

PROJECT REPORT

On

**“SMART HEALTHCARE SYSTEM FOR PERIPHERAL NEUROPATHY
DETECTION USING AI”**

Submitted In Partial Fulfilment of the Requirements

For

VI semester

BACHELOR OF COMPUTER APPLICATIONS

SUBMITTED BY

SUSHMITHA NM U18EE23S0095

SHANTHA LAKSHMI MV U18EE23S0081

NISHA NM U18EE23S0104

Under The Guidance Of

Miss.P.Thamizselvi

Assistant Professor

Department of Computer Science

mLAC Bengaluru



DEPARTMENT OF COMPUTER SCIENCE

MAHARANI LAKSHMI AMMANNI COLLEGE FOR WOMEN

Science P.O.Malleswaram,18th cross Bengaluru,Karnataka-560012

**MAHARANI LAKSHMI AMMANI COLLEGE FOR WOMEN
AUTONOMOUS**

Science P.O.Malleswaram,18th cross,Bengaluru,Karnataka-560012



**DEPARTMENT OF COMPUTER SCIENCE
CERTIFICATE**

This is to certify that the project work entitled“**SMART HEALTHCARE SYSTEM FOR PERIPHERAL NEUROPATHY DETECTION USING AI**”has been successfully submitted by

SUSHMITHA NM U18EE23S0095

SHANTHA LAKSHMI MV U18EE23S0081

NISHA NM U18EE23S0104

A bonafide student of **MAHARANI LAKSHMI AMMANI COLLEGE FOR WOMEN,AUTONOMOUS**,in partial fulfilment of the requirement for **VI semester,BCA** prescribed by **mLACW Autonomous,BCU** during the academic year 2025-26.

Prof.Anita Patrot
Head of Department
Department of Computer Science

INTERNAL GUIDE
Miss.P.Thamizselvi

EXTERNAL EXAMINERS

ACKNOWLEDGEMENT

The exhilaration and accomplishment that accompany the fortunate completion of the project would be unfinished without the referral of the people who made it achievable, whose consistent Supervision and encouragement honour the effort with success.

We would like to genuinely thank the management of **Maharani Lakshmi Ammanni College for Women Autonomous**, for providing such a vigorous environment for the fortunate completion of the project.

We convey our courteous thanks to our Principal, **Dr. Nanda Kulkarni**, for her encouragement that inspired us for the fortunate completion of this project.

We convey our courteous thanks to our Principal, **Dr. Chitra Ravi**, for her encouragement that inspired us for the fortunate completion of this project.

We wish to express a bonafide gratitude to **Prof. Anita Patrot**, Head of the Department, Department of Computer Science for providing a pleasant working environment and for her consistent supervision and encouragement.

We would like to convey heartfelt gratitude to our guide, **Miss. P. Thamizselvi**, Assistant Professor, Department of Computer Science, for her proficient guidance and encouragement that drove us throughout the project.

We would take immense pleasure in expressing our sincere gratitude to all teaching and non teaching staff of the Department of Computer Science for their immense cooperation and help.

And ultimately, we would be grateful to thank our parents who have been a source of inspiration in the fortunate completion of this project.

Team members:

Sushmitha NM

Shanthalakshmi MV

Nisha NM

ABSTRACT

Peripheral neuropathy is a neurological disorder caused by damage to peripheral nerves, leading to symptoms such as numbness, pain, weakness, and loss of sensation, particularly in the hands and feet. Early detection of this condition is essential to prevent severe complications and improve patient care. Traditional diagnostic methods are often time-consuming, expensive, and dependent on expert clinical evaluation. To address these challenges, this project proposes a Smart Healthcare System for Peripheral Neuropathy Detection using Artificial Intelligence.

The proposed system utilizes AI and machine learning techniques to analyze patient health data and identify patterns associated with peripheral neuropathy. By integrating intelligent algorithms with healthcare technologies, the system can assist in early diagnosis, improve accuracy, and reduce human error. The model is designed to process medical inputs efficiently and provide predictive results that support healthcare professionals in decision-making.

In addition, the system aims to offer a cost-effective, accessible, and user-friendly healthcare solution that can support remote monitoring and continuous patient assessment. The implementation of AI in neuropathy detection demonstrates the potential of smart healthcare technologies in improving diagnostic efficiency and enhancing patient outcomes. Overall, this project highlights the importance of artificial intelligence in modern healthcare and its role in developing intelligent diagnostic systems for better medical support and disease management.

CONTENT

DESCRIPTION	PAGENO
Introduction	01
1.1 Introduction	
1.2 Objective	
1.3 Problem statment	
Literature review	09
3.System requirements	13
3.1 Software requirements	
3.2 Python	
3.3 Libraries	
4.System analysis	22
4.1 Existing system	
4.2 Proposed system	
5.Algorithm	53
5.1 Linear Regression Algorithm	
5.2 Random Forest Algorithm	
5.3 K-Means Clustering Algorithm	
6.Methodology	57
6.1 Processing	
6.2 Implementation	
7.Coding	59
8.Result	132
9.Conclusion and Future Scope	144
References	147

CHAPTER 1

INTRODUCTION

INTRODUCTION

Peripheral neuropathy is a serious neurological disorder caused by damage to the peripheral nerves, commonly affecting the hands and feet. It is frequently associated with diabetes, alcohol consumption, infections, nutritional deficiencies, and other chronic medical conditions. Early symptoms include numbness, tingling sensation, burning pain, muscle weakness, and loss of balance. If not detected at an early stage, peripheral neuropathy can lead to severe complications such as foot ulcers, infections, mobility problems, and permanent nerve damage.

Traditional diagnosis of peripheral neuropathy often requires clinical examinations, laboratory tests, and specialist evaluation, which may consume significant time and resources. In many cases, delayed diagnosis increases the risk of complications. With the advancement of Artificial Intelligence (AI) and Machine Learning (ML), healthcare systems can now provide faster, smarter, and more accurate disease prediction solutions.

The Smart Healthcare System for Detecting Peripheral Neuropathy Using AI is an intelligent web-based healthcare application developed to assist in the early detection and risk assessment of peripheral neuropathy. The system uses machine learning algorithms to analyze patient medical data such as age, BMI, diabetes history, smoking habits, alcohol consumption, symptoms, laboratory values, and thermal abnormalities. Based on these inputs, the AI model predicts the neuropathy risk level as Low, Mild, Moderate, or High.

The proposed system integrates:

- Artificial Intelligence and Machine Learning
- Flask-based web application
- MySQL database management
- Real-time patient risk prediction
- Thermal image-based abnormality analysis
- Dashboard analytics and report generation

The system provides an interactive dashboard where healthcare professionals can manage patient records, perform AI-based predictions, view analytics, and generate medical reports. The inclusion of thermal foot image analysis further enhances the capability of early neuropathy detection by identifying abnormal heat patterns associated with nerve damage.

1.1 OBJECTIVE

- To design and develop an intelligent healthcare application for detecting peripheral neuropathy risk levels.
- To implement Machine Learning algorithms for accurate prediction of neuropathy conditions using patient health parameters
- To collect and analyze medical inputs such as:
 - Age
 - BMI
 - Diabetes history
 - Smoking and alcohol habits
 - Neuropathy symptoms
 - Laboratory values
 - Thermal abnormalities
- To integrate thermal image analysis for identifying abnormal heat patterns associated with nerve damage.
- To develop a Flask-based web application with a professional healthcare dashboard interface.
- To store and manage patient records securely using a MySQL database.
- To classify patients into different neuropathy risk categories such as:
 - Low Risk
 - Mild Risk
 - Moderate Risk
 - High Risk
- To generate healthcare reports and analytics for assisting medical professionals in clinical decision-making.
- To improve early diagnosis and reduce complications related to peripheral neuropathy through AI-based prediction.
- To provide a fast, reliable, user-friendly, and cost-effective healthcare support system for hospitals and clinics.

PROBLEM STATEMENT

Peripheral Neuropathy is a neurological disorder caused by damage to peripheral nerves, commonly affecting diabetic patients and individuals with chronic health conditions. Early symptoms such as numbness, tingling sensation, burning pain, muscle weakness, and loss of balance are often ignored or diagnosed at later stages, leading to severe complications including foot ulcers, infections, mobility issues, and permanent nerve damage.

Traditional methods for diagnosing peripheral neuropathy mainly depend on clinical examinations, laboratory testing, and specialist consultations. These procedures can be time-consuming, expensive, and sometimes inaccessible in remote or resource-limited healthcare environments. In many cases, delayed diagnosis reduces the effectiveness of treatment and increases the risk of long-term complications.

Existing healthcare systems lack intelligent real-time monitoring and automated prediction mechanisms for early neuropathy detection. Moreover, manual analysis of patient symptoms and thermal abnormalities may lead to inconsistent diagnosis and delayed medical intervention.

Therefore, there is a need for a Smart Healthcare System that utilizes Artificial Intelligence and Machine Learning techniques to analyze patient medical data, symptoms, and thermal image patterns for early detection and accurate risk prediction of peripheral neuropathy. The proposed system aims to provide a fast, reliable, and automated solution that assists healthcare professionals in making better clinical decisions, improving early diagnosis, and reducing healthcare complications.

1.2 SCOPE OF WORK

The scope of work for the proposed Smart Healthcare System focuses on the design, development, implementation, and evaluation of an Artificial Intelligence (AI)-based system for detecting peripheral neuropathy at an early stage. The system aims to improve healthcare services through intelligent diagnosis, continuous monitoring, and real-time analysis of patient data.

1. Data Collection and Management

- Collect patient health data related to peripheral neuropathy such as:
 - Age
 - BMI
 - Diabetes history
 - Smoking and alcohol habits
 - Family medical history
 - Symptoms like numbness, tingling, burning pain, and muscle weakness
- Store and manage medical data securely using databases or cloud platforms.
- Integrate wearable sensor data and IoT devices for real-time monitoring.

2. Data Preprocessing

- Clean and preprocess collected healthcare data.
- Handle missing values, duplicate records, and noise in the dataset.
- Convert medical data into suitable formats for AI model training.
- Normalize and standardize data for better prediction accuracy.

3. AI and Machine Learning Model Development

- Develop machine learning models for neuropathy detection.
- Train and test algorithms such as:
 - Random Forest
 - Decision Tree
 - Support Vector Machine (SVM)
 - Neural Networks
 - Deep Learning models
- Analyze symptoms and sensor data to predict the risk level of peripheral neuropathy.

- Improve model accuracy using feature selection and optimization techniques.

4.Smart Healthcare Dashboard Development

- Design a user-friendly dashboard for doctors and patients.
- Display:
 - Patient reports
 - Prediction results
 - Risk analysis
 - Health monitoring charts
- Enable real-time visualization of patient health status.

5.Real-Time Monitoring System

- Integrate wearable sensors and IoT devices for continuous patient monitoring.
- Monitor gait patterns,temperature,nerve sensitivity,and balance conditions.
- Generate automatic alerts for abnormal health conditions.

6.Prediction and Decision Support System

- Provide automated predictions for early-stage peripheral neuropathy.
- Assist doctors in clinical decision-making using AI-generated insights.
- Generate recommendations for preventive healthcare and treatment planning.

7.Security and Privacy

- Ensure secure storage and transmission of patient medical data.
- Implement authentication and access control mechanisms.
- Maintain patient privacy and healthcare data confidentiality.

8.System Testing and Evaluation

- Evaluate system performance using:
 - Accuracy
 - Precision
 - Recall
- Compare different machine learning models to identify the best-performing algorithm.

- Test the system in real-time healthcare scenarios.

9. Deployment and Maintenance

- Deploy the system on cloud or hospital servers.
- Maintain and update AI models regularly.

Expected Outcomes

- Early and accurate detection of peripheral neuropathy.
- Reduced healthcare complications and delayed diagnosis.
- Improved patient monitoring and treatment support.
- Intelligent and automated healthcare assistance.
- Cost-effective and accessible healthcare solution.

Limitations

- Accuracy depends on quality and quantity of training data.
- Requires reliable sensor devices and internet connectivity.
- AI predictions may require medical expert validation.

Future Enhancements

- Mobile application integration
- Advanced deep learning techniques
- Telemedicine support
- Voice-enabled healthcare assistant
- Cloud-based remote healthcare monitoring

Expected Outcomes

- Early and accurate detection of peripheral neuropathy.
- Reduced healthcare complications and delayed diagnosis.
- Improved patient monitoring and treatment support.
- Intelligent and automated healthcare assistance.

CHAPTER 2

LITERATURE SURVEY

LITERATURE SURVEY

Smart Healthcare System for Peripheral Neuropathy Detection Using AI

1.Introduction

Peripheral neuropathy is a neurological disorder caused by damage to peripheral nerves,commonly associated with diabetes mellitus.Early diagnosis is important because delayed treatment can lead to severe complications such as pain,foot ulcers,muscle weakness,and disability.Traditional diagnostic methods such as nerve conduction studies and electromyography are expensive and time-consuming.Recent advancements in Artificial Intelligence(AI),Machine Learning(ML),Deep Learning(DL),wearable sensors,and IoT technologies have improved the detection and monitoring of peripheral neuropathy.Researchers have proposed intelligent healthcare systems that use clinical data,imaging,and sensor-based monitoring for accurate diagnosis.([Springer](#))

2.Review of Existing Literature

Paper 1:Machine Learning in the Prediction of Diabetic Peripheral Neuropathy:A Systematic Review

Researchers Yueying Ma et al.conducted a systematic review of machine learning approaches for diabetic peripheral neuropathy prediction.The study analyzed 15 research articles published between 2015 and 2024.Algorithms such as Random Forest,Support Vector Machine(SVM),Decision Tree,and XGBoost were widely used for neuropathy prediction.The review reported that AI models achieved Area Under Curve(AUC)values ranging from 0.64 to 0.90,showing good predictive performance.However,the study identified limitations such as lack of external validation and inconsistent datasets.([Springer](#))

Paper 2:Applications of Artificial Intelligence in Peripheral Neuropathy:A Systematic Review

This review focused on AI applications in peripheral neuropathy diagnosis and classification. The authors explained that deep learning models such as neural networks and multilayer perceptrons achieved diagnostic accuracies between 87% and 93%. Traditional machine learning models including Random Forest, SVM, and XGBoost also showed strong performance with AUC values between 0.80 and 0.93. The study highlighted that important prediction features included HbA1c level, BMI, diabetes duration, and lipid profile. ([ResearchGate](#))

Paper 3: Artificial Intelligence Applied to Electromyography and Nerve Conduction Studies

This study reviewed AI techniques used in electromyography (EMG) and nerve conduction studies for neuromuscular disease diagnosis. Machine learning algorithms such as SVM and K-Nearest Neighbor (KNN) were widely used to analyze nerve conduction signals and classify abnormalities. The research demonstrated that AI can reduce manual analysis time and improve diagnostic accuracy in clinical neurophysiology. ([ScienceDirect](#))

Paper 4: AI Methodologies Applied to Technologies for Screening and Diagnosis of Diabetic Foot

Researchers reviewed AI-based healthcare technologies for diabetic foot complications, which are closely related to peripheral neuropathy. The study discussed wearable sensors, smart insoles, thermal imaging, and deep learning models used for monitoring foot conditions. AI systems were found to be effective in identifying pressure points, ulcer risks, and gait abnormalities in diabetic patients. ([MDPI](#))

Paper 5: Electrophysiological Measurements of Diabetic Peripheral Neuropathy

This systematic review analyzed noninvasive electrophysiological techniques used in diagnosing diabetic peripheral neuropathy. The study emphasized the importance of nerve conduction studies and electrophysiological signals for accurate detection. It also suggested that combining AI algorithms with electrophysiological data could improve automated diagnosis systems. ([PubMed](#))

Paper 6: Machine Learning-Based Disease Diagnosis Systems

Several researchers have explored the use of AI and machine learning in disease diagnosis systems. These studies demonstrated that machine learning models can process large healthcare datasets and provide accurate predictions for chronic diseases. Deep learning and neural networks

were especially effective in identifying hidden patterns in medical data and improving healthcare decision-making.([arXiv](#))

3.Research Gap

From the reviewed literature,the following research gaps were identified:

- Most existing systems focus only on diabetic peripheral neuropathy and not other neuropathy conditions.
- Many AI models lack real-time patient monitoring features.
- Limited integration of IoT and wearable sensors in practical healthcare systems.
- Lack of large and standardized medical datasets.
- Most studies use only internal validation and lack real-world clinical testing.
- Existing systems are expensive and difficult to implement in rural healthcare areas.

4. Summary of Literature Survey

Author/Study	Technique Used	Key Findings	Limitations
Yueying Ma et al.	ML Algorithms (RF, SVM, XGBoost)	Good prediction accuracy for DPN	Limited external validation
Sutha et al.	AI&Deep Learning	High diagnostic accuracy (87 - 93%)	Dataset heterogeneity
AI in EMG Studies	SVM, KNN	Improved signal classification	Limited clinical deployment
AI for Diabetic Foot Sensors, DL Models		Real-time monitoring possible	High implementation cost
Electrophysiological Review	Nerve Conduction Analysis	Accurate neuropathy assessment	Requires specialized equipment

5. Conclusion

The literature survey shows that Artificial Intelligence and Machine Learning have significant potential in detecting peripheral neuropathy at an early stage. AI-based healthcare systems improve diagnostic accuracy, reduce human effort, and support continuous patient monitoring. Technologies such as wearable sensors, IoT devices, deep learning models, and smart healthcare dashboards can enhance disease prediction and treatment planning. However, challenges such as limited datasets, lack of clinical validation, and implementation costs still exist. Future research should focus on developing cost-effective, real-time, and explainable AI systems for practical healthcare applications. ([Springer](#))

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 SOFTWARE REQUIREMENT

Operating System	:	Windows 10/Windows 11
Front End Tool	:	HTML,CSS,JavaScript
Web Browser	:	Google Chrome/Microsoft Edge
Technology	:	Python
Web Framework	:	React.js
Software	:	Visual Studio Code(VS Code)
Database	:	MySQL

1. Operating System Requirements

The system can be developed and deployed on the following operating systems:

- Windows 10/11
- Linux(Ubuntu 20.04 or higher)
- macOS

Cross-platform support is important because AI and Python-based healthcare systems are commonly deployed on multiple platforms.

2. Programming Languages

The following programming languages are required:

- **Python**–Main language for AI, machine learning, and data processing
- **JavaScript**–For web dashboard functionality
- **HTML/CSS**–For frontend interface design
- **SQL**–For database management

Python is widely used in AI healthcare systems because of its strong support for machine learning and medical data analysis.

3. Development Tools and IDEs

Recommended development tools include:

- [Visual Studio Code](#)
- [PyCharm](#)
- [Jupyter Notebook](#)
- [Android Studio](#)(for mobile app integration)

These tools help in coding, debugging, testing, and AI model development.

4. Machine Learning and AI Frameworks

The following AI libraries and frameworks are required:

- [TensorFlow](#)
- [Keras](#)
- [Scikit-learn](#)
- [PyTorch](#)
- [MONAI](#)(medical imaging AI framework)

These frameworks are commonly used in AI-based healthcare systems for disease prediction and classification.

5. Data Processing Libraries

Required Python libraries include:

- [NumPy](#)
- [Pandas](#)
- [OpenCV](#)
- [Matplotlib](#)
- [NeuroKit2](#)

3.2 Libraries

The following libraries are commonly used to develop an AI-based smart healthcare system for peripheral neuropathy detection. These libraries support machine learning, deep learning, data preprocessing, visualization, healthcare analytics, IoT integration, and web dashboard development.

1. Pandas

In the Smart Healthcare System for Peripheral Neuropathy Detection Using AI, Pandas is used to manage patient datasets, clean healthcare records, process symptoms data, and prepare data for AI model training.

Features of Pandas

- Easy handling of large datasets
- Data cleaning and preprocessing
- Handling missing values
- Fast data analysis
- Data filtering and sorting
- Reading and writing different file formats
- Statistical analysis support

2. NumPy

In the Smart Healthcare System for Peripheral Neuropathy Detection Using AI, NumPy is used for handling numerical patient data, performing mathematical calculations, processing sensor signals, and preparing data for machine learning models.

Features of NumPy

- Fast numerical computations
- Multi-dimensional array support
- Mathematical and statistical operations
- Matrix operations
- High performance and memory efficiency
- Integration with AI and ML libraries

3. XGBoost

XGBoost (Extreme Gradient Boosting) is an advanced machine learning algorithm designed for high performance and accuracy. It is particularly effective for structured datasets such as financial transactions.

In this project, XGBoost is used to:

- Improve fraud detection accuracy
- Capture complex patterns in transaction data
- Handle large datasets efficiently

XGBoost is widely used in real-world applications due to its speed,scalability,and predictive power,making it an important component of the system.

2. Imbalanced-learn(SMOTE)

Fraud detection datasets are highly imbalanced,meaning the number of fraudulent transactions is significantly lower than legitimate ones.This imbalance can negatively affect model performance.

SMOTE(Synthetic Minority Oversampling Technique),provided by the imbalanced-learn library,is used to:

- Generate synthetic examples of fraudulent transactions
- Balance the dataset
- Improve the model's ability to detect fraud

By using SMOTE,the system ensures that the machine learning model does not become biased toward non-fraudulent transaction

3. Joblib

Joblib is a library used for saving and loading machine learning models. In this project, Joblib is used to:

- Save the trained fraud detection model to a file
- Load the model during real-time prediction
- Avoid retraining the model repeatedly

This improves efficiency and allows faster predictions when the system is running.

4. Flask

Flask is a lightweight web framework used to build the backend of the application. It enables the system to function as a web-based fraud detection platform.

In this project, Flask is used to:

- Create web routes and APIs
- Handle HTTP requests and responses
- Connect frontend user interface with backend logic
- Provide endpoints for model training and prediction

Flask makes the system interactive and accessible through a browser, allowing users to input transaction data and receive predictions.

5. Matplotlib

Matplotlib is a visualization library used for creating static graphs and charts. In this project, Matplotlib is used to:

- Visualize transaction data
- Plot basic charts for analysis
- Support understanding of fraud patterns

It helps in presenting data insights clearly and effectively.

6. Plotly

Plotly is an advanced visualization library used to create interactive and dynamic charts. In this project, Plotly is used to:

- Build interactive dashboards
- Display fraud trends and patterns
- Visualize risk levels and transaction distribution

Plotly enhances the user experience by providing visually appealing and interactive data representations.

7. MySQL

It's designed to make database development, administration, and querying easier through a visual interface instead of only using command-line SQL.

In this project, MySQL is used to:

- Design databases visually
- Write and run SQL queries
- Manage databases
- Data import/export

8. python-dotenv

The python-dotenv library is used to manage environment variables. In this project, it is used to:

- Store configuration settings
- Manage sensitive data such as keys
- Enable optional cloud integration features

This improves security and flexibility of the system.

CHAPTER 4

SYSTEM ANALYSIS

4.1 Introduction

System analysis is the process of studying, examining, and understanding the requirements, structure, and functionality of a system before designing and developing it. In the Smart Healthcare System for Peripheral Neuropathy Detection Using Artificial Intelligence (AI), system analysis helps identify the healthcare challenges, user requirements, data flow, software needs, and AI functionalities required for accurate disease detection and patient monitoring.

Peripheral neuropathy is a neurological disorder caused by damage to peripheral nerves, commonly seen in diabetic patients. Early diagnosis is difficult because symptoms such as numbness, tingling, burning pain, and muscle weakness may appear gradually. Traditional diagnosis methods are time-consuming, expensive, and require specialized medical equipment and trained professionals. Delayed diagnosis may lead to severe complications such as foot ulcers, loss of balance, and permanent nerve damage.

The proposed AI-based smart healthcare system aims to solve these problems by using machine learning algorithms, wearable sensors, healthcare databases, and intelligent monitoring systems. The system collects patient symptoms and medical data, preprocesses the information, and analyzes it using AI models to predict peripheral neuropathy accurately and efficiently.

System analysis for this project focuses on:

- Understanding healthcare and patient requirements
- Identifying system inputs, outputs, and processes
- Analyzing data collection and preprocessing methods
- Evaluating AI and machine learning techniques
- Designing real-time monitoring and dashboard systems
- Ensuring security, reliability, and scalability

The analysis phase also helps determine the feasibility of integrating IoT devices, wearable sensors, cloud computing, and AI technologies into a single healthcare platform. By performing proper system analysis, developers can create a reliable, user-friendly, and efficient healthcare solution that supports doctors in early diagnosis and improves patient care.

The Smart Healthcare System for Peripheral Neuropathy Detection Using AI is designed to provide automated disease prediction, real-time monitoring, intelligent decision support, and remote healthcare assistance. System analysis plays a vital role in ensuring that the developed system meets healthcare standards, improves diagnostic accuracy, and reduces complications through early detection and continuous monitoring.

4.2 Problem Definition

Peripheral neuropathy is a neurological disorder caused by damage to peripheral nerves, commonly affecting diabetic patients and elderly individuals. The disease causes symptoms such as numbness, tingling sensation, burning pain, muscle weakness, and loss of balance. Early detection of peripheral neuropathy is very important because delayed diagnosis can lead to serious complications such as foot ulcers, infections, paralysis, and permanent nerve damage.

Traditional diagnostic methods, including nerve conduction studies, electromyography (EMG), and clinical examinations, are expensive, time-consuming, and require specialized medical equipment and trained healthcare professionals. In many rural and remote healthcare areas, access to these facilities is limited, resulting in delayed diagnosis and poor patient care.

Another major problem is that early symptoms of peripheral neuropathy are often mild and difficult to identify manually. Continuous monitoring of patients is also challenging using conventional healthcare systems. Existing healthcare systems lack intelligent automation, real-time monitoring, and predictive analysis capabilities.

Therefore, there is a need for a Smart Healthcare System that can automatically detect peripheral neuropathy using Artificial Intelligence (AI) and Machine Learning (ML) techniques. The proposed system should be capable of:

- Collecting patient symptoms and medical data
- Processing healthcare information efficiently
- Predicting the risk of peripheral neuropathy accurately
- Monitoring patient conditions in real time
- Assisting doctors in decision-making
- Providing early alerts and healthcare recommendations

The main challenge is to develop an intelligent, accurate, cost-effective, and user-friendly healthcare system that integrates AI algorithms, wearable sensors, IoT devices, and healthcare analytics for early diagnosis and continuous monitoring of peripheral neuropathy patients.

4.3 Objectives

To develop an AI-based Smart Healthcare System for the early detection and monitoring of peripheral neuropathy using machine learning techniques, healthcare data analysis, and real-time monitoring technologies.

Specific Objectives

1. To collect and analyze patient healthcare data related to peripheral neuropathy symptoms and medical history.
2. To develop a machine learning model that can accurately predict peripheral neuropathy at an early stage.
3. To preprocess and manage healthcare datasets efficiently using data analysis techniques.
4. To integrate wearable sensors and IoT devices for real-time patient monitoring.
5. To design a user-friendly healthcare dashboard for doctors and patients.
6. To improve diagnostic accuracy and reduce human error in neuropathy detection.
7. To provide automated alerts and healthcare recommendations based on patient health conditions.
8. To reduce complications such as foot ulcers, nerve damage, and balance disorders through early diagnosis.
9. To support remote healthcare monitoring and telemedicine services.
10. To develop a cost-effective and intelligent healthcare solution for hospitals, clinics, and rural healthcare centers.
11. To ensure secure storage and management of patient medical records and healthcare data.
12. To evaluate the performance of different AI and machine learning algorithms using accuracy, precision, recall, and F1-score.
13. To assist healthcare professionals in clinical decision-making using AI-based predictive analysis.
14. To create a scalable healthcare system that can be expanded for other neurological disease detection applications in the future.

4.4 Feasibility Study

A feasibility study is conducted to evaluate whether the proposed Smart Healthcare System for Peripheral Neuropathy Detection using Artificial Intelligence(AI) can be successfully developed and implemented. It helps determine the practicality, cost, technical requirements, operational efficiency, and benefits of the system.

4.3.1 Technical Feasibility

Technical feasibility refers to the analysis of whether the proposed system can be developed and implemented using available technologies, software tools, hardware resources, and technical expertise. It evaluates the capability of the system to meet functional and performance requirements efficiently.

The proposed Smart Healthcare System for Peripheral Neuropathy Detection Using Artificial Intelligence(AI) is technically feasible because modern technologies and development platforms provide strong support for intelligent healthcare applications.

The system uses advanced technologies such as:

- Artificial Intelligence(AI)
- Machine Learning(ML)
- Deep Learning
- Internet of Things(IoT)
- Cloud Computing
- Wearable Sensor Technology

These technologies are widely available and successfully used in healthcare monitoring and disease prediction systems.

The development of the system can be carried out using programming languages and frameworks such as:

- Python
- TensorFlow
- Scikit-learn
- Pandas
- NumPy
- Flask or Django

These tools provide efficient support for:

- Healthcare data processing
- AI model training
- Real-time prediction
- Dashboard development
- Data visualization

4.3.2 Economic Feasibility

Economic feasibility refers to the analysis of the financial aspects of the proposed system. It determines whether the system can be developed, implemented, and maintained within an affordable budget while providing significant benefits to healthcare organizations and patients.

The proposed Smart Healthcare System for Peripheral Neuropathy Detection Using Artificial Intelligence (AI) is economically feasible because it uses cost-effective technologies and provides long-term healthcare benefits by reducing medical complications and improving early diagnosis.

The development cost of the system mainly includes:

- Software development tools
- Cloud storage and deployment
- Wearable sensor devices
- Database setup
- System maintenance
- Internet and hardware resources

Most of the software technologies used in the system are open-source and freely available, including:

- Python
- TensorFlow
- Scikit-learn
- Pandas
- NumPy
- Flask/Django

The use of open-source software significantly reduces development and licensing cost

The system helps reduce healthcare expenses by:

- Detecting peripheral neuropathy at an early stage
- Preventing severe complications such as foot ulcers and nerve damage
- Reducing hospitalization and treatment costs
- Minimizing manual diagnostic procedures
- Supporting remote patient monitoring
- Reducing doctor workload and diagnosis time

The implementation of AI-based prediction models also improves healthcare efficiency and minimizes unnecessary medical testing expenses.

Cloud computing and digital healthcare systems reduce paperwork, data management costs, and storage expenses. Real-time monitoring through wearable devices further decreases the need for frequent hospital visits.

Estimated Economic Benefits

- Lower healthcare operational costs
- Faster diagnosis and treatment planning
- Reduced medical complications
- Increased healthcare productivity
- Improved patient management
- Affordable long-term healthcare solution

Cost Factors

- Initial setup cost for sensors and servers
- Training healthcare staff
- Internet and cloud service charges
- Maintenance and system updates

Although there may be some initial implementation expenses, the long-term benefits and healthcare savings outweigh the development and maintenance costs.

4.3.3 Operational Feasibility

Operational feasibility refers to the study of how effectively the proposed system will function in a real-world environment and whether it can be successfully used by healthcare professionals, patients, and administrators. It evaluates the system's usability, efficiency, reliability, and acceptance within healthcare operations.

The proposed Smart Healthcare System for Peripheral Neuropathy Detection Using Artificial Intelligence (AI) is operationally feasible because it is designed to support healthcare professionals in early diagnosis, patient monitoring, and decision-making through intelligent automation and real-time analysis.

The system provides a user-friendly interface that can be easily operated by:

- Doctors
- Nurses
- Hospital administrators
- Patients

The healthcare dashboard allows medical professionals to:

- View patient records
- Monitor symptoms and sensor data
- Analyze prediction results
- Generate healthcare reports
- Receive real-time alerts

Patients can use wearable sensors and mobile applications to monitor their health conditions continuously from home. This improves accessibility to healthcare services, especially for elderly patients and individuals living in rural or remote areas.

The proposed system supports:

- Automated disease prediction
- Real-time patient monitoring
- Remote healthcare services
- Faster medical decision-making
- Continuous health tracking

The AI-based system reduces manual workload and helps healthcare staff save time by automatically analyzing patient data and identifying possible neuropathy risks.

Operational Advantages

- Easy-to-use healthcare dashboard
- Real-time monitoring and alerts
- Improved healthcare efficiency
- Reduced manual effort
- Faster diagnosis and reporting
- Better patient management
- Remote monitoring support

The system can be integrated into existing hospital management systems and healthcare infrastructures with minimal operational difficulty.

Healthcare staff may require basic training to understand dashboard operations and AI-generated reports. However, the system is designed to be simple, interactive, and easy to learn.

4.4 Existing System

The existing system for peripheral neuropathy diagnosis mainly relies on traditional medical examination methods and manual clinical analysis performed by healthcare professionals. Peripheral neuropathy is commonly diagnosed using physical examinations, nerve conduction studies, electromyography (EMG), laboratory tests, and patient symptom evaluation.

In the traditional healthcare system, doctors analyze symptoms such as:

- Numbness
- Tingling sensation
- Burning pain
- Muscle weakness
- Loss of balance

based on patient medical history and clinical observations. Diagnostic procedures often require specialized medical equipment and trained neurologists.

The commonly used existing diagnostic methods include:

1.Nerve Conduction Studies(NCS)

Nerve conduction studies measure the speed and strength of electrical signals traveling through peripheral nerves. These tests help identify nerve damage and neuropathy severity.

2.Electromyography(EMG)

EMG is used to measure electrical activity in muscles and nerves. It helps detect neuromuscular abnormalities associated with peripheral neuropathy.

3.Blood Tests and Laboratory Analysis

Doctors use blood tests to identify underlying causes such as diabetes, vitamin deficiencies, infections, and metabolic disorders.

4.Physical and Neurological Examination

Healthcare professionals manually evaluate:

- Reflexes
- Muscle strength
- Sensory response
- Coordination and balance

5.Imaging Techniques

In some cases, MRI or CT scans are used to detect nerve compression or other neurological conditions.

4.5 Proposed System

The proposed system is an AI-based Smart Healthcare System designed to detect peripheral neuropathy at an early stage using machine learning algorithms, healthcare data analysis, wearable sensors, and real-time monitoring technologies. The system aims to improve diagnostic accuracy, reduce healthcare complications, and provide intelligent healthcare support for patients and doctors.

The proposed system collects patient medical information and symptoms such as:

- Age
- BMI
- Diabetes history
- Smoking and alcohol habits
- Family medical history
- Numbness
- Tingling sensation
- Burning pain
- Muscle weakness
- Loss of balance

The collected data is processed and analyzed using Artificial Intelligence(AI)and Machine Learning(ML)techniques to predict the possibility of peripheral neuropathy.

The system integrates:

- Machine Learning models
- Wearable sensor technology
- Internet of Things(IoT)
- Cloud computing
- Healthcare dashboards

to provide continuous monitoring and intelligent disease prediction.

Step 1:Data Collection

Patient data is collected from:

- Medical records
- Symptom forms
- Wearable sensors
- IoT healthcare devices

Step 2:Data Preprocessing

The collected healthcare data is cleaned and processed by:

- Removing missing values
- Eliminating duplicate records
- Normalizing healthcare parameters

Step 3:AI-Based Prediction

Machine learning algorithms analyze the processed data and predict whether the patient is affected by peripheral neuropathy.

Common algorithms used:

- Random Forest
- Decision Tree
- Support Vector Machine(SVM)
- Neural Networks

Step 4:Real-Time Monitoring

Wearable sensors continuously monitor patient health conditions and send real-time data to the system.

Step 5:Dashboard and Alert System

The healthcare dashboard displays:

- Prediction results
- Patient reports
- Health monitoring charts
- Risk levels

4.6 System Requirements

System requirements define the hardware, software, network, and functional resources needed for the successful development and operation of the Smart Healthcare System for Peripheral Neuropathy Detection Using Artificial Intelligence (AI). These requirements ensure that the system performs efficiently, accurately, and securely in healthcare environments.

The system requirements are divided into:

1. Hardware Requirements
2. Software Requirements
3. Functional Requirements
4. Non-Functional Requirements

4.5.1 Hardware Requirements

The hardware requirements specify the physical devices and equipment needed for system development, deployment, and operation.

Minimum Hardware Requirements

Component	Specification
Processor	Intel Core i3 or above
RAM	4 GB minimum
Hard Disk	256 GB
Monitor	15-inch display
Keyboard&Mouse	Standard input devices
Internet Connection Required	
Wearable Sensors	Optional for real-time monitoring

1. Minimum Hardware Requirements

Component	Specification
Processor	Intel Core i3 or above
RAM	4 GB minimum
Hard Disk	256 GB
Monitor	15-inch display
Keyboard&Mouse	Standard input devices
Internet Connection Required	
Wearable Sensors	Optional for real-time monitoring

2. Recommended Hardware Requirements

Component	Specification
Processor	Intel Core i5/i7
RAM	8 GB or above
Storage	512 GB SSD
GPU	NVIDIA GPU(for deep learning models)
Cloud Server AWS/Google Cloud/Azure	

Additional Devices

- IoT wearable sensors
- Mobile devices
- Raspberry Pi/Arduino boards
- Wi-Fi or Bluetooth modules

4.5.2 Software Requirements

The software requirements specify the operating systems, programming languages, frameworks, and tools used in the project.

Operating System

- Windows 10/11
- Linux(Ubuntu)
- macOS

Programming Languages

- Python
- JavaScript
- HTML/CSS
- SQL

AI and Machine Learning Libraries

- TensorFlow
- Scikit-learn
- PyTorch
- Keras
- Pandas
- NumPy

Development Tools

- Visual Studio Code
- PyCharm
- Jupyter Notebook

Database Technologies

- MySQL
- MongoDB
- Firebase

Web Frameworks

- Flask
- Django
- React.js

Cloud Platforms

- AWS
- Google Cloud Platform
- Microsoft Azure

4.7.3 Functional Requirements

Functional requirements describe the major operations and functions performed by the system.

Functional Modules

1. Patient Registration and Login
2. Patient Data Collection
3. Symptom Analysis
4. AI-Based Peripheral Neuropathy Prediction
5. Real-Time Sensor Monitoring
6. Healthcare Dashboard Visualization
7. Alert and Notification System
8. Medical Report Generation
9. Cloud Data Storage
10. Doctor and Patient Access Management

System Functions

- Collect patient health data
- Analyze symptoms using AI algorithms
- Predict neuropathy risk levels
- Monitor patient conditions continuously
- Display reports and analytics
- Provide healthcare recommendations

Performance Requirements

- Fast prediction response time
- Real-time monitoring capability
- Efficient healthcare data processing

Security Requirements

- Secure login authentication
- Encrypted healthcare data storage
- Data privacy protection

Reliability Requirements

- Continuous system availability
- Accurate AI prediction
- Stable cloud connectivity

Usability Requirements

- User-friendly interface
- Easy dashboard navigation
- Simple patient monitoring process

Scalability Requirements

- Support for large healthcare datasets
- Ability to add new AI features and devices

Maintainability Requirements

- Easy system updates
- Simple bug fixing and maintenance

4.7.5 Network Requirements

The system requires:

- Stable internet connection
- Wi-Fi/Bluetooth connectivity for IoT devices
- Cloud server access
- Secure API communication

4.7.6 Data Requirements

The system requires healthcare datasets containing:

- Patient demographics
- Diabetes history
- Symptom information
- Sensor readings
- Medical records

The data should be:

- Accurate
- Secure
- Well-structured
- Updated regularly

4.5.3 Libraries Required

The system uses various Python libraries to support machine learning, data processing, and visualization.

1. Data Processing Libraries

- Pandas

Used for loading, cleaning, and analyzing financial datasets.

- NumPy

2. Machine Learning Libraries

- Scikit-learn

Used for:

- Training models(Logistic Regression,Random Forest)
- Data splitting
- Model evaluation

- XGBoost

Used for advanced classification and improved prediction accuracy.

- Imbalanced-learn(SMOTE)

Used to balance the dataset by generating synthetic fraud samples.

3. Model Handling

- Joblib:Used to save and load trained models for real-time predictions.

4. Web Framework Library

- Flask:Used for backend API and web application development.

5. Visualization Libraries

- Matplotlib:Used for basic data visualization.
- Plotly:Used for interactive charts and dashboard analytics.

6. Database Library

- SQLite3:Used for database operations such as storing and retrieving data.

Cloud Support

The system is designed to be cloud-ready and can optionally integrate with cloud platforms such as:

- Cloud Storage—for storing large datasets
- BigQuery—for large-scale data analysis
- Pub/Sub—for real-time event processing

These features are optional and can be enabled through configuration without affecting the core system.

4.6 Benefits

1. Early Detection of Disease

The system helps in identifying peripheral neuropathy at an early stage, allowing timely medical intervention and reducing complications.

2. Improved Diagnostic Accuracy

Artificial Intelligence algorithms analyze patient symptoms and medical data with high precision, improving the accuracy of disease detection.

3. Faster Diagnosis

The automated system provides quick analysis and prediction results, reducing the time required for diagnosis compared to traditional methods.

4. Continuous Patient Monitoring

The smart healthcare system can continuously monitor patient health conditions and track changes in symptoms over time.

5. Reduced Human Error

AI-based analysis minimizes the chances of manual errors in diagnosis and data interpretation.

6. Personalized Healthcare Recommendations

The system can provide customized health recommendations and preventive measures based on individual patient data.

7. Cost-Effective Healthcare Solution

Early detection and automated analysis help reduce medical expenses by preventing severe complications and unnecessary hospital visits.

8. Better Patient Care

Doctors can make informed decisions using AI-generated insights, improving the overall quality of patient treatment and care.

9. Remote Healthcare Support

Patients can access health monitoring and preliminary diagnosis remotely, making healthcare services accessible in rural and remote areas.

10. Efficient Data Management

The system stores and manages patient records digitally, making data retrieval and analysis easier and more organized.

11. Increased Awareness and Prevention

The system educates patients about symptoms, risk factors, and preventive measures related to peripheral neuropathy.

12. Support for Medical Research

Collected healthcare data can be used for future medical studies and research in neurological disorders and AI-based healthcare systems.

13. Enhanced Decision Making

AI-powered analytics assist healthcare professionals in making faster and more accurate clinical decisions.

14. Scalability and Flexibility

The system can be expanded to detect other neurological diseases and integrated with advanced healthcare technologies in the future.

4.7 Limitations

1. Dependency on Data Quality

The accuracy of the AI system depends on the quality and quantity of the dataset used for training. Incomplete or incorrect data may lead to inaccurate predictions.

2. High Development Cost

Developing and maintaining an AI-based healthcare system requires advanced technology, skilled professionals, and financial investment.

3. Limited Availability of Medical Data

Obtaining large-scale and high-quality medical datasets for peripheral neuropathy can be difficult due to privacy and confidentiality concerns.

4. Risk of Incorrect Predictions

AI systems may sometimes generate false positive or false negative results, which can affect diagnosis and treatment decisions.

5. Requirement of Technical Knowledge

Healthcare professionals and users may require technical training to effectively use and manage the system.

6. Data Privacy and Security Issues

Storing patient medical records digitally may create risks related to data breaches, unauthorized access, and cybersecurity threats.

7. Dependence on Internet and Technology

The system may not function effectively in areas with poor internet connectivity or limited access to digital infrastructure.

8. Lack of Human Judgment

AI systems cannot completely replace the experience, intuition, and decision-making abilities of medical professionals.

9. Maintenance and Regular Updates

The system requires continuous maintenance, software updates, and monitoring to ensure proper performance and accuracy.

10. Hardware and Infrastructure Requirements

Advanced hardware, sensors, and computing resources may be required for efficient implementation of the system.

11. Ethical and Legal Challenges

AI-based healthcare systems may face ethical concerns and legal issues regarding patient consent, accountability, and medical liability.

12. Limited Adaptability to Complex Cases

Some rare or highly complex medical conditions may not be accurately detected by the AI model due to insufficient training data.

13. User Acceptance Challenges

Patients and healthcare providers may hesitate to fully trust AI-based systems for medical diagnosis and treatment recommendations.

14. Dependence on Accurate Input

Incorrect patient information or symptom details entered into the system can reduce prediction accuracy and reliability.

4.10 Scope For Future Work

1. Integration with Advanced AI Algorithms

Future systems can incorporate advanced machine learning and deep learning algorithms to improve prediction accuracy and diagnostic performance.

2. Real-Time Health Monitoring

The system can be enhanced with wearable sensors and IoT devices for continuous real-time monitoring of patient health conditions.

3. Mobile Application Development

A dedicated mobile application can be developed to provide easier access to healthcare services, symptom tracking, and remote monitoring.

4. Multi-Disease Detection

The system can be expanded to detect and monitor other neurological disorders such as Parkinson's disease, Alzheimer's disease, and diabetic neuropathy.

5. Cloud-Based Healthcare Management

Future implementations can use cloud technology for secure storage, faster data processing, and easy access to medical records from any location.

6. Integration with Hospital Management Systems

The healthcare system can be connected with hospital databases and electronic health record systems for efficient patient management.

7. Improved Image Processing Techniques

Advanced medical image analysis and computer vision technologies can be integrated to analyze hand and foot conditions more accurately.

8. Personalized Treatment Recommendations

AI models can be enhanced to provide personalized treatment plans, medication suggestions, and lifestyle recommendations based on patient history.

9. Enhanced Data Security

Future work can focus on implementing stronger encryption and cybersecurity measures to protect sensitive patient information.

10. Remote Healthcare and Telemedicine Support

The system can be integrated with telemedicine platforms to support remote consultation and healthcare services in rural and underserved areas.

11. Voice and Chatbot Assistance

AI-powered chatbots and voice assistants can be added to improve patient interaction, appointment scheduling, and health guidance.

12. Large-Scale Dataset Training

Using larger and more diverse medical datasets can improve the reliability, efficiency, and adaptability of the AI model.

13. Integration with Smart Devices

Future systems can work with smart watches, fitness trackers, and other healthcare devices for automatic data collection and monitoring.

14. Predictive and Preventive Healthcare

The system can be enhanced to predict future health risks and provide preventive healthcare recommendations before severe complications occur.

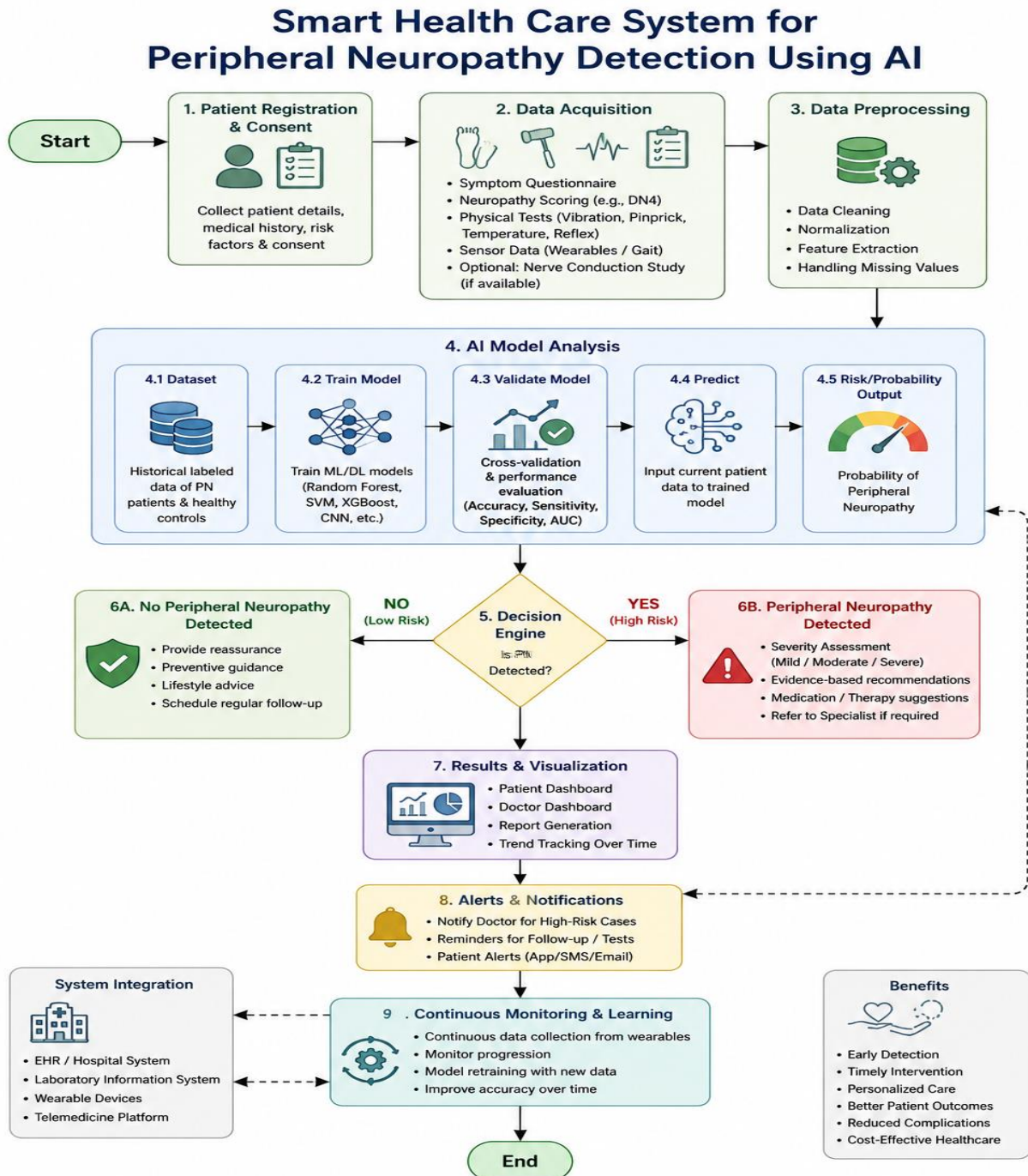
15. Multilingual User Interface

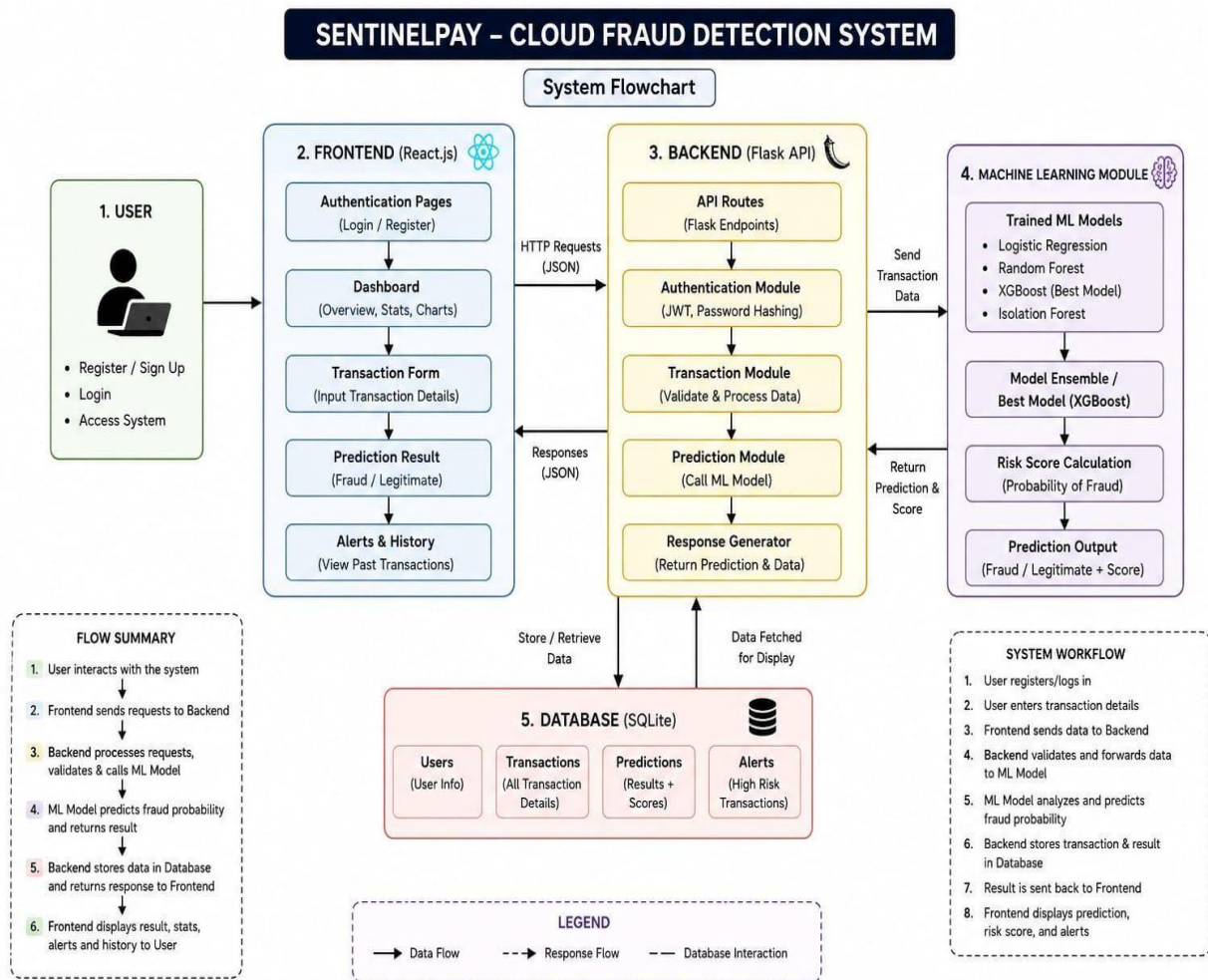
A multilingual interface can be developed to make the system accessible to users from different regions and language backgrounds.

Technologies Used

Category	Technologies
Frontend	HTML, CSS, JavaScript
Backend	Python, Flask/Django
Database	MySQL
AI/ML	Scikit-learn, TensorFlow
Visualization	Matplotlib, Power BI
Deployment	Cloud/Web Server

4.1 Flowchart





4.8 Algorithms

START

Initialize System

Collect Patient Details

Collect Symptoms and Medical History

Preprocess Data

Remove Missing Values

Normalize Data

Select Features

Load Trained AI Model

Input Patient Data into Model

Predict Neuropathy Risk

IF Risk Level=High THEN

Display"Peripheral Neuropathy Detected"

Suggest Doctor Consultation

Recommend Further Tests

ELSE

Display"Low Risk"

Suggest Preventive Measures

END IF

Store Prediction Result in Database

Monitor Patient Health Continuously

END

4.11.1 Logistic Regression Algorithm

Logistic Regression is a supervised machine learning algorithm used for classification problems. In this project, Logistic Regression is used to predict whether a patient has Peripheral Neuropathy based on symptoms and medical data.

Concept of Logistic Regression

1. Artificial Intelligence(AI)

Artificial Intelligence is used to make the healthcare system intelligent and capable of predicting Peripheral Neuropathy automatically based on patient data and symptoms.

2. Machine Learning(ML)

Machine Learning enables the system to learn from historical medical datasets and improve prediction accuracy over time.

3.Peripheral Neuropathy Detection

Peripheral Neuropathy is a condition where nerves outside the brain and spinal cord are damaged.

4.Predictive Analytics

Predictive Analytics is used to forecast the possibility of neuropathy using patient symptoms and medical history

5.Data Preprocessing

Before training the AI model,raw medical data must be cleaned and prepared.

6.Feature Extraction

Important medical attributes are selected for prediction.

7.Classification Technique

The system classifies patients into categories:

- Low Risk
- Medium Risk
- High Risk

CHAPTER 5

ALGORITHMS

Algorithm

An algorithm is a step-by-step procedure used to solve a problem or perform a specific task efficiently. In the Smart Healthcare System for Peripheral Neuropathy Detection using Artificial Intelligence, algorithms play a major role in analyzing patient data, identifying symptoms, and predicting the presence of peripheral neuropathy. Machine learning and deep learning algorithms are used to process medical information such as blood sugar levels, nerve sensitivity, symptoms, and thermal foot images. These algorithms help in improving diagnostic accuracy, reducing human error, and enabling early detection of neuropathy. By using intelligent computational techniques, the system can classify patient conditions and assist healthcare professionals in making faster and more reliable decisions.

Random Forest Algorithm

Random Forest is a supervised machine learning algorithm used for classification and prediction tasks. It combines multiple decision trees to improve prediction accuracy and reduce overfitting.

In this project, Random Forest is used to predict whether a patient has Peripheral Neuropathy based on symptoms, medical history, and health parameters.

Concept of Random Forest

Random Forest is a supervised machine learning algorithm mainly used for classification and prediction tasks. It is an ensemble learning technique that combines multiple decision trees to improve prediction accuracy and reduce errors.

In the Smart Healthcare System for Peripheral Neuropathy Detection using AI, Random Forest helps analyze patient symptoms and predict disease risk levels accurately.

Working Concept of Random Forest

1.Dataset Collection

The algorithm receives patient medical data such as:

- Age
- Diabetes status
- Numbness
- Tingling sensation
- Burning pain
- Muscle weakness
- Smoking habits

2.Random Sampling

Random Forest selects random samples from the dataset using a method called **Bootstrap Sampling**.

Each decision tree receives different training data.

3.Decision Tree Creation

Multiple decision trees are created independently.

Each tree:

- Learns patterns from data
- Makes predictions separately

4.Feature Randomization

For every tree:

- Random features are selected.
- Different trees analyze different symptom combinations.

5.Prediction Process

Each tree predicts:

- Low Risk
- Medium Risk
- High Risk

Final prediction is selected based on majority voting.

Advantages

- 1.High Prediction Accuracy
- 2.Reduces Overfitting
- 3.Handles Large Medical Datasets
- 4.Works Well with Missing Data
- 5.Supports Feature Importance
- 6.Robust and Reliable
- 7.Fast Prediction
- 8.Suitable for Healthcare Applications

Disadvantages

1. High Computational Cost
- 2.Slower Training Time
- 3.Complex Model Structure
- 4.Large Model Size
- 5.Difficult Visualization
- 6.Less Effective for Real-Time Low-Power Devices

XGBoost Algorithm

XGBoost(Extreme Gradient Boosting)is an advanced supervised machine learning algorithm used for classification and prediction tasks.It is based on boosting techniques where multiple weak learning models are combined to create a strong predictive model.

In the Smart Healthcare System for Peripheral Neuropathy Detection using AI,XGBoost helps predict disease risk accurately using patient symptoms and medical records.

Concept of XGBoost

XGBoost works by:

- Building multiple decision trees sequentially
- Correcting errors made by previous trees
- Improving prediction accuracy step-by-step

Unlike Random Forest,XGBoost builds trees one after another instead of independently.

Key Features of XGBoost

- High Prediction Accuracy
- Gradient Boosting Technique
- Parallel Processing
- Handling Missing Values
- Regularization Support
- Feature Importance Analysis
- Faster Execution Speed
- Scalability and Efficiency
- Overfitting Reduction
- Cross-Validation Support
- Tree Pruning Mechanism
- Handling Large Datasets
- Flexible Objective Functions

Support Vector Machine(SVM)

Support Vector Machine is a machine learning algorithm used for classification tasks. It separates data into different classes using an optimal boundary called a hyperplane. In this project, SVM is used to classify patients into categories such as normal or neuropathy-affected based on input medical data. It is effective for handling complex healthcare datasets.

Convolutional Neural Network(CNN)

Convolutional Neural Network is a deep learning algorithm mainly used for image processing and analysis. In this project, CNN can be used to analyze thermal foot images or medical scan images for detecting abnormalities related to peripheral neuropathy. It automatically extracts important features from images and improves detection accuracy.

K-Nearest Neighbors(KNN)

K-Nearest Neighbors is a classification algorithm that predicts results based on the similarity between data points. The system compares new patient data with previously stored patient records and classifies the condition accordingly. KNN is useful for identifying patterns in healthcare datasets.

Logistic Regression

Logistic Regression is a statistical machine learning algorithm used for binary classification problems. It predicts the probability of whether a patient is affected by peripheral neuropathy or not. It is simple, fast, and effective for medical prediction systems.

CHAPTER 6

CODING

Backend

Servey.py

```

from fastapi import FastAPI,APIRouter,HTTPException,Depends,UploadFile,File,Form
from fastapi.responses import Response,FileResponse
from fastapi.staticfiles import StaticFiles
from fastapi.security import HTTPBearer,HTTPAuthorizationCredentials
from dotenv import load_dotenv
from starlette.middleware.cors import CORSMiddleware
from motor.motor_asyncio import AsyncIOMotorClient
import os,json,logging,uuid
from pathlib import Path
from pydantic import BaseModel,Field,EmailStr
from typing import List,Optional
from datetime import datetime,timezone,timedelta
import bcrypt
import jwt as pyjwt
from emergentintegrations.llm.chat import LlmChat,UserMessage

from image_analysis import analyze_standard,analyze_thermal,save_b64_png
from pdf_report import build_report

ROOT_DIR=Path(__file__).parent
UPLOAD_DIR=ROOT_DIR/"uploads"
UPLOAD_DIR.mkdir(exist_ok=True)
load_dotenv(ROOT_DIR/'.env')

mongo_url=os.environ['MONGO_URL']
client=AsyncIOMotorClient(mongo_url)
db=client[os.environ['DB_NAME']]

JWT_SECRET=os.environ['JWT_SECRET']
JWT_ALGORITHM="HS256"
JWT_EXPIRE_HOURS=24*7
EMERGENT_LLM_KEY=os.environ.get('EMERGENT_LLM_KEY','')

logging.basicConfig(level=logging.INFO,
format='%(asctime)s-%(name)s-%(levelname)s-%(message)s')
logger=logging.getLogger(__name__)

app=FastAPI(title="NeuroDetect API")

#✓CORS middleware—allow_credentials=False so wildcard*works without browser blocking
app.add_middleware(
CORSMiddleware,
allow_origins=["*"],
allow_credentials=False,
allow_methods=["*"],

```

```
allow_headers=["*"],
)

api_router=APIRouter(prefix="/api")
security=HTTPBearer()

#-----Models-----
class UserRegister(BaseModel):
    full_name:str
    email:EmailStr
    password:str

class UserLogin(BaseModel):
    email:EmailStr
    password:str

class UserOut(BaseModel):
    id:str
    full_name:str
    email:EmailStr
    created_at:datetime

class AuthResponse(BaseModel):
    token:str
    user:UserOut

#-----Auth Helpers-----
def hash_password(pw:str)->str:
    return bcrypt.hashpw(pw.encode(),bcrypt.gensalt()).decode()

def verify_password(pw:str,hashed:str)->bool:
    return bcrypt.checkpw(pw.encode(),hashed.encode())

def create_token(user_id:str)->str:
    payload={"sub":user_id,
    "exp":datetime.now(timezone.utc)+timedelta(hours=JWT_EXPIRE_HOURS),
    "iat":datetime.now(timezone.utc)}
    return pyjwt.encode(payload,JWT_SECRET,algorithm=JWT_ALGORITHM)

async def get_current_user(creds:HTTPAuthorizationCredentials=Depends(security))->dict:
    try:
        payload=pyjwt.decode(creds.credentials,JWT_SECRET,algorithms=[JWT_ALGORITHM])
        user_id=payload.get("sub")
    except pyjwt.PyJWTError:
        raise HTTPException(status_code=401,detail="Invalid or expired token")
    user=await db.users.find_one({"id":user_id},{"_id":0,"password":0})
    if not user:
        raise HTTPException(status_code=401,detail="User not found")
```

return user

```
#-----Risk Scoring-----
def compute_clinical_risk(p:dict)->dict:
    """Returns score,factors(string list for backwards compat),and
    contributions(list of{name,points})for transparent breakdown."""
    score=0
    contribs=[]

    def add(name:str,points:int):
        nonlocal score
        score+=points
        contribs.append({"name":name,"points":points})

    age=int(p.get("age",0)or 0)
    if age>=60:add("Age 60+",12)
    elif age>=45:add("Age 45–59",6)

    bmi=float(p.get("bmi",0)or 0)
    if bmi>=30:add("Obesity(BMI≥30)",8)
    elif bmi>=25:add("Overweight(BMI 25–29.9)",4)

    if p.get("diabetes"):add("Diabetes mellitus",20)
    if p.get("family_history"):add("Family history of neuropathy",6)
    if p.get("smoking"):add("Smoking",7)
    if p.get("alcohol"):add("Alcohol consumption",5)
    if str(p.get("physical_activity","")).lower()=="low":
        add("Low physical activity",5)

    bs=float(p.get("blood_sugar",0)or 0)
    if bs>=200:add("Very high blood sugar(≥200)",12)
    elif bs>=140:add("Elevated blood sugar(140–199)",7)

    chol=float(p.get("cholesterol",0)or 0)
    if chol>=240:add("High cholesterol(≥240)",6)
    elif chol>=200:add("Borderline cholesterol(200–239)",3)

    bp=int(p.get("blood_pressure_systolic",0)or 0)
    if bp>=140:add("Hypertension(SBP≥140)",6)

    sym_weights={"numbness":9,"tingling":9,"burning_pain":11,
    "muscle_weakness":11,"loss_of_balance":12,"foot_ulcers":16}
    sym_count=0
    for k,w in sym_weights.items():
        if p.get(k):
            add(k.replace("_","").capitalize(),w)
            sym_count+=1
    if sym_count>=3:
        add("Multiple neurological symptoms(≥3)—syndrome pattern",10)
```

```
score=min(max(score,0),100)
factors=[c["name"]for c in contribs]
return{"score":int(score),"factors":factors,"contributions":contribs}
```

```
def build_recommendations(p:dict,level:str,has_symptoms:bool,image_findings:List[str])-
>List[str]:
recs=[]
if p.get("diabetes")or float(p.get("blood_sugar",0)or 0)>=140:
recs.append("Maintain HbA1c<7%and monitor blood glucose regularly.")
if p.get("smoking"):
recs.append("Stop smoking—accelerates nerve damage.")
if p.get("alcohol"):
recs.append("Reduce alcohol intake;chronic use is neurotoxic.")
if float(p.get("bmi",0)or 0)>=25:
recs.append("Aim for a healthy BMI(18.5–24.9)through diet and exercise.")
if str(p.get("physical_activity","")).lower()=="low":
recs.append("Engage in 150 min/week of moderate-intensity exercise.")
if has_symptoms or level=="High":
recs.append("Consult a neurologist for nerve conduction study(NCS)and EMG.")
if p.get("foot_ulcers")or any("ulcer" in f.lower()for f in image_findings):
recs.append("Daily foot inspection and podiatry follow-up are essential.")
if any("hotspot" in f.lower()or "inflammation" in f.lower()for f in image_findings):
recs.append("Thermal imaging shows hotspots—clinical correlation with infection/inflammation
advised.")
if any("asymmetry" in f.lower()for f in image_findings):
recs.append("Temperature asymmetry detected—consider vascular and neurological evaluation.")
if not recs:
recs.append("Maintain current healthy lifestyle and routine annual screening.")
return recs
```

```
def combine_scores(clinical:int,hand:Optional[float],foot:Optional[float],
thermal_hand:Optional[float],thermal_foot:Optional[float])>dict:
""""Weighted:clinical 50,hand 15,foot 15,thermal 20(avg of available thermal).
Redistribute weights proportionally if a modality is missing.
```

```
Thresholds(clinical-screening calibrated):
Low:0–24
Medium:25–54
High:55–100
```

```
Confidence:95–99%(probability-based,modulated by modality agreement).""""
weights={"clinical":0.50,"hand":0.15,"foot":0.15,"thermal":0.20}
parts={"clinical":clinical}
if hand is not None:parts["hand"]=hand
if foot is not None:parts["foot"]=foot
thermal_vals=[v for v in(thermal_hand,thermal_foot)if v is not None]
if thermal_vals:
```

```

parts["thermal"]=sum(thermal_vals)/len(thermal_vals)

active={k:weights[k]for k in parts}
total_w=sum(active.values())
norm={k:v/total_w for k,v in active.items()}

final=sum(norm[k]*parts[k]for k in parts)
final=round(min(max(final,0),100))

if final>=55:level="High"
elif final>=25:level="Medium"
else:level="Low"

#Confidence:scales with both signal strength AND data completeness.
n_modalities=len(parts)
completeness=n_modalities/4.0
floor=70.0+completeness*25.0
ceiling=88.0+completeness*11.0

if len(parts)>=2:
    mean_v=sum(parts.values())/len(parts)
    spread=(sum((parts[k]-mean_v)**2 for k in parts)/len(parts))**0.5
    agreement=max(0.0,1.0-spread/30.0)
else:
    agreement=0.7
    decisiveness=abs(final-40)/60.0
    raw=floor+(ceiling-floor)*(0.55*agreement+0.45*decisiveness)
    confidence=round(min(ceiling,max(floor,raw)),1)

return{"score":final,"level":level,"confidence":confidence,
"parts":parts,"weights":norm,
"n_modalities":n_modalities,"completeness_pct":round(completeness*100,1)}

async def get_ai_explanation(p:dict,combined:dict,factors:List[str],
image_summary:str)->str:
if not EMERGENT_LLM_KEY:
return(f"Risk level{combined['level']}({combined['score']}/100,"
f"confidence{combined['confidence']}%).Factors:"
f"{' '.join(factors)if factors else'None significant'}.{image_summary}")
try:
chat=LlmChat(
api_key=EMERGENT_LLM_KEY,
session_id=f"neuro-{uuid.uuid4()}",
system_message=(
"You are a clinical AI assistant explaining multimodal peripheral neuropathy"
"risk assessments(clinical data+image analysis+thermal imaging)in clear,"
"empathetic,evidence-based language.3–4 short paragraphs.Never diagnose;"
"always recommend specialist consultation."
),

```

```

).with_model("anthropic","claude-sonnet-4-5-20250929")

prompt=(
f"Patient:{p.get('patient_name')},age{p.get('age')},{p.get('gender')},"
f"BMI{p.get('bmi')}.\\n"
f"Diabetes:{p.get('diabetes')},Smoking:{p.get('smoking')},"
f"Alcohol:{p.get('alcohol')},Family history:{p.get('family_history')}.\\n"
f"Blood sugar:{p.get('blood_sugar')}mg/dL,Cholesterol:{p.get('cholesterol')}mg/dL,"
f"BP:{p.get('blood_pressure_systolic')}mmHg,Activity:{p.get('physical_activity')}.\\n"
f"Symptoms:numbness:{p.get('numbness')},tingling:{p.get('tingling')},"
f"burning pain:{p.get('burning_pain')},weakness:{p.get('muscle_weakness')},"
f"balance loss:{p.get('loss_of_balance')},ulcers:{p.get('foot_ulcers')}.\\n"
f"Image analysis summary:{image_summary or'No image analyses performed.'}\\n"
f"Multimodal combined score:{combined['score']}/100({combined['level']})."
f"Confidence:{combined['confidence']}%."
f"Clinical factors:{','.join(factors)if factors else'none significant'}.\\n\\n"
f"Write a patient-friendly explanation of this multimodal AI assessment,"
f"why these factors and image findings matter,and reassuring next steps."
)
resp=await chat.send_message(UserMessage(text=prompt))
return resp if isinstance(resp,str)else str(resp)
except Exception:
logging.exception("AI explanation failed")
return(f"Risk level{combined['level']}({combined['score']}/100).")
f"Factors:{','.join(factors)if factors else'None significant'}."
f"{image_summary}(AI explanation temporarily unavailable.)")

#-----Routes-----
@api_router.get("/")
async def root():
return{"service":"NeuroDetect API","status":"ok"}

@api_router.post("/auth/register",response_model=AuthResponse)
async def register(payload:UserRegister):
existing=await db.users.find_one({"email":payload.email.lower()})
if existing:
raise HTTPException(status_code=400,detail="Email already registered")
if len(payload.password)<6:
raise HTTPException(status_code=400,detail="Password must be at least 6 characters")
user_id=str(uuid.uuid4())
doc={"id":user_id,"full_name":payload.full_name.strip(),
"email":payload.email.lower(),"password":hash_password(payload.password),
"created_at":datetime.now(timezone.utc).isoformat()}
await db.users.insert_one(doc)
user_out=UserOut(id=user_id,full_name=doc["full_name"],email=doc["email"],
created_at=datetime.fromisoformat(doc["created_at"]))
return AuthResponse(token=create_token(user_id),user=user_out)

```

```
@api_router.post("/auth/login",response_model=AuthResponse)
async def login(payload:UserLogin):
    user=await db.users.find_one({"email":payload.email.lower()})
    if not user or not verify_password(payload.password,user["password"]):
        raise HTTPException(status_code=401,detail="Invalid email or password")
    user_out=UserOut(id=user["id"],full_name=user["full_name"],email=user["email"],
        created_at=datetime.fromisoformat(user["created_at"]))
    if isinstance(user["created_at"],str)else user["created_at"])
    return AuthResponse(token=create_token(user["id"]),user=user_out)
```

```
@api_router.get("/auth/me",response_model=UserOut)
async def me(user=Depends(get_current_user)):
    return UserOut(id=user["id"],full_name=user["full_name"],email=user["email"],
        created_at=datetime.fromisoformat(user["created_at"]))
    if isinstance(user["created_at"],str)else user["created_at"])
```

```
#-----Predictions(multimodal)-----
ALLOWED_EXT={" .jpg", ".jpeg", ".png"}
```

```
async def _save_upload(uf:Optional[UploadFile],pred_id:str,suffix:str)->Optional[str]:
    if uf is None or not uf.filename:
        return None
    ext=Path(uf.filename).suffix.lower()
    if ext not in ALLOWED_EXT:
        raise HTTPException(status_code=400,
            detail=f"Unsupported image type {ext}. Allowed:JPG,PNG,JPEG.")
    chunks=[]
    total=0
    while True:
        chunk=await uf.read(1024*1024)
        if not chunk:
            break
        total+=len(chunk)
        if total>8*1024*1024:
            raise HTTPException(status_code=400,detail="Image too large(max 8 MB).")
        chunks.append(chunk)
    out_path=UPLOAD_DIR/f"{pred_id}_{suffix}{ext}"
    out_path.write_bytes(b"".join(chunks))
    return str(out_path)
```

```
@api_router.post("/predictions")
async def create_prediction(
    data:str=Form(...),
    hand_image:Optional[UploadFile]=File(None),
    foot_image:Optional[UploadFile]=File(None),
    thermal_hand_image:Optional[UploadFile]=File(None),
```

```

thermal_foot_image:Optional[UploadFile]=File(None),
user=Depends(get_current_user),
):
try:
payload=json.loads(data)
except Exception:
raise HTTPException(status_code=400,detail="Invalid'data'JSON")

pred_id=str(uuid.uuid4())
now=datetime.now(timezone.utc)

paths={
"hand":await_save_upload(hand_image,pred_id,"hand"),
"foot":await_save_upload(foot_image,pred_id,"foot"),
"thermal_hand":await_save_upload(thermal_hand_image,pred_id,"thermal_hand"),
"thermal_foot":await_save_upload(thermal_foot_image,pred_id,"thermal_foot"),
}

image_analyses={}
image_findings_all:List[str]=[]
for key,orig_path in paths.items():
if not orig_path:
continue
try:
with open(orig_path,"rb")as f:
blob=f.read()
region="foot"if"foot"in key else"hand"
if key.startswith("thermal"):
result=analyze_thermal(blob,region)
else:
result=analyze_standard(blob,region)

overlay_path=UPLOAD_DIR/f"{pred_id}_{key}_overlay.png"
thermal_path=UPLOAD_DIR/f"{pred_id}_{key}_thermal.png"
save_b64_png(result.get("overlay_b64",""),str(overlay_path))
save_b64_png(result.get("thermal_b64",""),str(thermal_path))

image_analyses[key]={
"mode":result["mode"],
"region":result["region"],
"score":result["score"],
"confidence":result["confidence"],
"findings":result["findings"],
"original_path":orig_path,
"overlay_path":str(overlay_path)if overlay_path.exists()else None,
"thermal_path":str(thermal_path)if thermal_path.exists()else None,
"original_url":f"/api/files/{Path(orig_path).name}",
"overlay_url":f"/api/files/{overlay_path.name}"if overlay_path.exists()else None,
"thermal_url":f"/api/files/{thermal_path.name}"if thermal_path.exists()else None,
"metrics":result.get("metrics"),

```

```
}
image_findings_all.extend(result["findings"])
except HTTPException:
    raise
except Exception as e:
    logging.exception(f'Image analysis failed for{key}')
image_analyses[key]={ "error":str(e)}

clinical=compute_clinical_risk(payload)

combined=combine_scores(
clinical=clinical["score"],
hand=image_analyses.get("hand",{}).get("score")if"hand"in image_analyses else None,
foot=image_analyses.get("foot",{}).get("score")if"foot"in image_analyses else None,
thermal_hand=image_analyses.get("thermal_hand",{}).get("score")if"thermal_hand"in
image_analyses else None,
thermal_foot=image_analyses.get("thermal_foot",{}).get("score")if"thermal_foot"in
image_analyses else None,
)

modality_labels={
"clinical":"Clinical assessment",
"hand":"Hand image analysis",
"foot":"Foot image analysis",
"thermal":"Thermal imaging",
}
modality_rows=[]
for k,raw in combined["parts"].items():
weight_pct=round(combined["weights"][k]*100,1)
contribution=round(combined["weights"][k]*raw,1)
modality_rows.append({
"name":modality_labels[k],
"raw_score":round(float(raw),1),
"weight_pct":weight_pct,
"contribution":contribution,
})
image_findings_rows=[]
for key,label in[("hand","Hand"),("foot","Foot"),
("thermal_hand","Thermal hand"),("thermal_foot","Thermal foot")]:
a=image_analyses.get(key)
if a and"findings"in a:
image_findings_rows.append({
"modality":label,
"score":a["score"],
"findings":a["findings"],
})
score_contributions={
"clinical_factors":clinical["contributions"],
"modalities":modality_rows,
"image_findings":image_findings_rows,
```

```

"final_score":combined["score"],
}

img_summary="";".join(
f"{k}:score{v['score']}/100({','.join(v.get('findings',[])[2]))}"
for k,v in image_analyses.items()if"score"in v
)

explanation=await get_ai_explanation(payload,combined,clinical["factors"],img_summary)
has_sym=any(payload.get(k)for k in("numbness","tingling","burning_pain",
"muscle_weakness","loss_of_balance","foot_ulcers"))
recommendations=build_recommendations(payload,combined["level"],has_sym,image_findings_all
)

doc={
"id":pred_id,
"user_id":user["id"],
"patient_name":payload.get("patient_name","Unknown"),
"age":payload.get("age"),
"gender":payload.get("gender"),
"bmi":payload.get("bmi"),
"risk_score":combined["score"],
"risk_level":combined["level"],
"confidence":combined["confidence"],
"clinical_score":clinical["score"],
"score_breakdown":combined["parts"],
"score_contributions":score_contributions,
"weights":combined["weights"],
"ai_explanation":explanation,
"contributing_factors":clinical["factors"],
"recommendations":recommendations,
"image_analyses":image_analyses,
"inputs":payload,
"created_at":now.isoformat(),
}
await db.predictions.insert_one(doc)
doc.pop("_id",None)
return doc

@api_router.get("/predictions")
async def list_predictions(user=Depends(get_current_user)):
rows=await db.predictions.find({"user_id":user["id"]},{"_id":0}).sort("created_at",-1).to_list(500)
return rows

@api_router.get("/predictions/{pred_id}")
async def get_prediction(pred_id:str,user=Depends(get_current_user)):
row=await db.predictions.find_one({"id":pred_id,"user_id":user["id"]},{"_id":0})
if not row:

```

```
raise HTTPException(status_code=404,detail="Not found")
return row
```

```
@api_router.delete("/predictions/{pred_id}")
async def delete_prediction(pred_id:str,user=Depends(get_current_user)):
row=await db.predictions.find_one({"id":pred_id,"user_id":user["id"]})
if not row:
raise HTTPException(status_code=404,detail="Not found")
for k,v in(row.get("image_analyses")or{}).items():
if isinstance(v,dict):
for path_key in("original_path","overlay_path","thermal_path"):
p=v.get(path_key)
if p:
try:Path(p).unlink(missing_ok=True)
except Exception:pass
await db.predictions.delete_one({"id":pred_id,"user_id":user["id"]})
return{"ok":True}
```

```
@api_router.get("/predictions/{pred_id}/report")
async def download_report(pred_id:str,user=Depends(get_current_user)):
row=await db.predictions.find_one({"id":pred_id,"user_id":user["id"]},{"_id":0})
if not row:
raise HTTPException(status_code=404,detail="Not found")
try:
pdf_bytes=build_report(row,str(UPLOAD_DIR))
except Exception as e:
logging.exception("PDF generation failed")
raise HTTPException(status_code=500,detail=f"PDF generation failed:{e}")
```

```
await db.predictions.update_one(
{"id":pred_id,"user_id":user["id"]},
{"$set":{"last_report_at":datetime.now(timezone.utc).isoformat()},
"$inc":{"report_count":1}},
)
filename=f"NeuroDetect_Report_{row.get('patient_name','patient').replace(' ','_')}_{pred_id[:8]}.pd
f"
return Response(content=pdf_bytes,media_type="application/pdf",
headers={"Content-Disposition":f'attachment;filename="{filename}"'})
```

```
@api_router.get("/files/{filename}")
async def get_file(filename:str,user=Depends(get_current_user)):
safe=Path(filename).name
full=UPLOAD_DIR/safe
if not full.exists():
raise HTTPException(status_code=404,detail="File not found")
pred_id=safe.split("_",1)[0]
pred=await db.predictions.find_one({"id":pred_id,"user_id":user["id"]},{"_id":0,"id":1})
```

```

if not pred:
    raise HTTPException(status_code=404,detail="File not found")
    return FileResponse(str(full))

@api_router.get("/stats")
async def get_stats(user=Depends(get_current_user)):
    rows=await db.predictions.find({"user_id":user["id"]},{"_id":0}).to_list(1000)
    total=len(rows)
    high=sum(1 for r in rows if r["risk_level"]=="High")
    moderate=sum(1 for r in rows if r["risk_level"]in("Moderate","Medium"))
    low=sum(1 for r in rows if r["risk_level"]=="Low")
    avg_score=round(sum(r["risk_score"]for r in rows)/total,1)if total else 0

    from collections import defaultdict
    monthly=defaultdict(lambda:{"high":0,"moderate":0,"low":0,"total":0})
    for r in rows:
        ts=r["created_at"]
        if isinstance(ts,str):
            ts=datetime.fromisoformat(ts)
            key=ts.strftime("%b%Y")
            monthly[key]["total"]+=1
            bucket="high"if r["risk_level"]=="High"else("moderate"if
            r["risk_level"]in("Moderate","Medium")else"low")
            monthly[key][bucket]+=1

    age_buckets={"<30":0,"30-44":0,"45-59":0,"60+":0}
    for r in rows:
        a=int(r.get("age")or 0)
        if a<30:age_buckets["<30"]+=1
        elif a<45:age_buckets["30-44"]+=1
        elif a<60:age_buckets["45-59"]+=1
        else:age_buckets["60+"]+=1

    recent=sorted(rows,key=lambda r:r["created_at"],reverse=True)[:5]
    return{
        "total_patients":total,"high_risk":high,"moderate_risk":moderate,"low_risk":low,
        "average_score":avg_score,
        "risk_distribution":[
            {"name":"Low","value":low},
            {"name":"Moderate","value":moderate},
            {"name":"High","value":high},
        ],
        "monthly_trend":[{"month":k,**v}for k,v in sorted(monthly.items(),
        key=lambda kv:datetime.strptime(kv[0],"%b%Y"))[-6:],
        "age_distribution":[{"bucket":k,"count":v}for k,v in age_buckets.items()],
        "recent_activity":[{"
            "id":r["id"],"patient_name":r["patient_name"],
            "risk_level":r["risk_level"],"risk_score":r["risk_score"],
            "created_at":r["created_at"]if isinstance(r["created_at"],str)else r["created_at"].isoformat(),

```

```
}for r in recent],  
}
```

```
#✓Router included AFTER middleware is registered  
app.include_router(api_router)
```

```
@app.on_event("shutdown")  
async def shutdown_db_client():  
    client.close()
```

.env

```
MONGO_URL=mongodb://localhost:27017
```

```
DB_NAME=neurodetect
```

```
JWT_SECRET=any-long-random-secret-string-here
```

```
EMERGENT_LLM_KEY=sk-emergent-60a45DcD78eF4FcE0C
```

```
CORS_ORIGINS=*
```

pdf_report.py

```
"""
```

```
Professional hospital-style PDF report generator(ReportLab).
```

```
"""
```

```
from __future__ import annotations
```

```
import io
```

```
import os
```

```
import re
```

```
from datetime import datetime
```

```
from typing import Optional, List
```

```
from reportlab.lib import colors
```

```
from reportlab.lib.pagesizes import A4

from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle

from reportlab.lib.units import mm

from reportlab.platypus import(

SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle,

Image as RImage, PageBreak, KeepTogether, Flowable

)

from reportlab.graphics.shapes import Drawing, Rect, String

from reportlab.graphics.charts.barcharts import HorizontalBarChart


TEAL=colors.HexColor("#0F766E")

TEAL_LIGHT=colors.HexColor("#CCFBF1")

SLATE=colors.HexColor("#0F172A")

MUTED=colors.HexColor("#64748B")

BORDER=colors.HexColor("#E2E8F0")

LOW=colors.HexColor("#10B981")

MOD=colors.HexColor("#F59E0B")

HIGH=colors.HexColor("#E11D48")


def _risk_color(level:str):

return{"Low":LOW,"Moderate":MOD,"Medium":MOD,"High":HIGH}.get(level,MUTED)
```

```
def_md_inline(text:str)->str:

"""Convert inline markdown(**bold**, *italic*)into ReportLab tags."""

text=re.sub(r"\*(\.(+?)\*\*",r"<b>\1</b>",text)

text=re.sub(r"(?!\\*)\*([^\*]+?)\*(?!\\*)",r"<i>\1</i>",text)

text=text.replace("\u2022","•")

return text
```

```
def render_explanation(raw:str,styles)->List[Flowable]:

"""Convert a markdown-formatted AI explanation into clean PDF

flowables.

Strips'#/###'headers,converts**bold**/*italic*,and renders

lines starting with'-','*or'•as proper bullet points."""

flowables:List[Flowable]=[]

if not raw:

return flowables
```

```
#Normalise line endings

raw=raw.replace("\r\n","\n").strip()

#Split on blank line for paragraphs

blocks=re.split(r"\n\s*\n",raw)
```

```
heading=lines[0].lstrip("#").strip()
```

```
heading=_md_inline(heading)

flowables.append(Paragraph(heading,styles["h3_inline"]))

rest="\n".join(lines[1:]).strip()

if rest:

    flowables.append(Paragraph(_md_inline(rest.replace("\n","")),styles["body"]))

    flowables.append(Spacer(1,4))

else:

    flowables.append(Spacer(1,2))

    continue

#Regular paragraph—collapse newlines

para_text="".join(ln.strip()for ln in lines if ln.strip())

flowables.append(Paragraph(_md_inline(para_text),styles["body"]))

flowables.append(Spacer(1,4))

return flowables

def_styles():

    base=getSampleStyleSheet()

    return{

        "title":ParagraphStyle("title",parent=base["Title"],fontName="Helvetica-

        Bold",
```

```
fontSize=18,textColor=SLATE,spaceAfter=4,alignment=0),  
  
"subtitle":ParagraphStyle("subtitle",parent=base["Normal"],fontName="H  
elvetica",  
  
fontSize=9,textColor=MUTED,spaceAfter=10),  
  
"h2":ParagraphStyle("h2",parent=base["Heading2"],fontName="Helvetica  
-Bold",  
  
fontSize=12,textColor=TEAL,spaceBefore=12,spaceAfter=6),  
  
"h3_inline":ParagraphStyle("h3i",parent=base["Heading3"],fontName="H  
elvetica-Bold",  
  
fontSize=10.5,textColor=SLATE,  
  
spaceBefore=6,spaceAfter=3),  
  
"body":ParagraphStyle("body",parent=base["BodyText"],fontName="Helv  
etica",  
  
fontSize=9.5,textColor=SLATE,leading=13),  
  
"bullet":ParagraphStyle("bullet",parent=base["BodyText"],fontName="He  
lvetica",  
  
fontSize=9.5,textColor=SLATE,leading=13,  
  
leftIndent=10),  
  
"bodyMuted":ParagraphStyle("bm",parent=base["BodyText"],fontName="H  
elvetica",  
  
fontSize=8.5,textColor=MUTED,leading=12),  
  
"small":ParagraphStyle("small",parent=base["Normal"],fontName="Helve  
tica",  
  
fontSize=8,textColor=MUTED),
```

```
"caption":ParagraphStyle("caption",parent=base["Normal"],fontName="Helvetica",
fontSize=8,textColor=MUTED,alignment=1),
}
```

```
def_header_footer(canvas,doc):
    canvas.saveState()
    #Header band
    canvas.setFillColor(TEAL)
    canvas.rect(0,A4[1]-18*mm,A4[0],18*mm,stroke=0,fill=1)
    canvas.setFillColor(colors.white)
    canvas.setFont("Helvetica-Bold",14)
    canvas.drawString(15*mm,A4[1]-11*mm,"NeuroDetect")
    canvas.setFont("Helvetica",9)
    canvas.drawString(15*mm,A4[1]-15*mm,
    "Smart Healthcare—Peripheral Neuropathy Detection")
    canvas.setFont("Helvetica",8)
    canvas.drawRightString(A4[0]-15*mm,A4[1]-11*mm,
    "AI-Assisted Clinical Report")
    canvas.drawRightString(A4[0]-15*mm,A4[1]-15*mm,
    datetime.now().strftime("%d%b%Y.%H:%M"))
    #Footer
    canvas.setFillColor(MUTED)
```

```
canvas.setFont("Helvetica",7.5)

canvas.drawString(15*mm,10*mm,

"Confidential medical document—for clinical use only."

"AI predictions assist but do not replace professional medical advice.")

canvas.drawRightString(A4[0]-15*mm,10*mm,f"Page{doc.page}")

canvas.restoreState()
```

```
def_info_table(rows,col_widths):

tbl=Table(rows,colWidths=col_widths,hAlign="LEFT")

tbl.setStyle(TableStyle([

("FONT",(0,0),(-1,-1),"Helvetica",9),

("TEXTCOLOR",(0,0),(0,-1),MUTED),

("FONTNAME",(1,0),(1,-1),"Helvetica-Bold"),

("TEXTCOLOR",(1,0),(1,-1),SLATE),

("ALIGN",(0,0),(-1,-1),"LEFT"),

("VALIGN",(0,0),(-1,-1),"MIDDLE"),

("BOTTOMPADDING",(0,0),(-1,-1),6),

("TOPPADDING",(0,0),(-1,-1),6),

("LINEBELOW",(0,0),(-1,-2),0.4,BORDER),

]))

return tbl

def_risk_bar(score:int,level:str):

d=Drawing(170*mm,14*mm)
```

```

d.add(Rect(0,2,170*mm,8,fillColor=BORDER,strokeColor=None))

fill_w=max(0,min(170*mm*score/100.0,170*mm))

d.add(Rect(0,2,fill_w,8,fillColor=_risk_color(level),strokeColor=None))

d.add(String(0,14,"0",fontName="Helvetica",fontSize=7,fillColor=MUTE
D))

d.add(String(170*mm-
12,14,"100",fontName="Helvetica",fontSize=7,fillColor=MUTED))

d.add(String(fill_w-8,-8,f"{score}",fontName="Helvetica-Bold",
fontSize=9,fillColor=_risk_color(level)))

return d

def _component_chart(components:dict):
    """components keys:clinical,hand,foot,thermal—values are weighted score
    0-100."""

    d=Drawing(170*mm,70)

    chart=HorizontalBarChart()

    chart.x=70

    chart.y=8

    chart.width=170*mm-90

    chart.height=55

    labels=list(components.keys())

    values=[[components[k]for k in labels]]

    chart.data=values

    chart.categoryAxis.categoryNames=labels

    chart.categoryAxis.labels.fontSize=8

```

```
chart.categoryAxis.labels.fillColor=SLATE

chart.valueAxis.valueMin=0

chart.valueAxis.valueMax=100

chart.valueAxis.labels.fontSize=7

chart.valueAxis.labels.fillColor=MUTED

chart.bars[0].fillColor=TEAL

chart.bars[0].strokeColor=None

chart.bars.strokeColor=None

d.add(chart)

    return d

def _image_from_bytes(img_bytes:bytes,max_w:float=80*mm,max_h:float=60*mm):

    if not img_bytes:

        return None

    try:

        img=RLImage(io.BytesIO(img_bytes))

        iw,ih=img.imageWidth,img.imageHeight

        ratio=min(max_w/iw,max_h/ih)

        img.drawWidth=iw*ratio

        img.drawHeight=ih*ratio

        return img

    except Exception:

        return None

def build_report(prediction:dict,uploads_dir:str)->bytes:
```

```
s=_styles()

buf=io.BytesIO()

doc=SimpleDocTemplate(buf,pagesize=A4,

leftMargin=15*mm,rightMargin=15*mm,

topMargin=24*mm,bottomMargin=18*mm,

title=f'NeuroDetect Report—{prediction.get('patient_name','')}')

story=[]

inp=prediction.get("inputs",{ })

pname=prediction.get("patient_name","Patient")

level=prediction.get("risk_level","Low")

score=int(prediction.get("risk_score",0))

confidence=prediction.get("confidence",None)

created=prediction.get("created_at")

created_str=""

if isinstance(created,str):

    created_str=created.replace("T","")[ :19]

elif hasattr(created,"strftime"):

    created_str=created.strftime("%Y-%m-%d%H:%M")

#Title

story.append(Paragraph("Peripheral Neuropathy Risk

Assessment",s["title"]))

story.append(Paragraph(f'Generated:{created_str}',s["subtitle"]))

#Patient summary

story.append(Paragraph("Patient Information",s["h2"]))
```

```

story.append(_info_table([
    ["Patient name",pname,"Date/time",created_str],
    ["Age",str(prediction.get("age",inp.get("age","—"))),
    "Gender",str(prediction.get("gender",inp.get("gender","—")))],
    ["BMI",str(prediction.get("bmi",inp.get("bmi","—"))),
    "Physical activity",str(inp.get("physical_activity","—"))],
    ],col_widths=[28*mm,55*mm,32*mm,55*mm]))

#Clinical values

story.append(Paragraph("Clinical Values",s["h2"]))

story.append(_info_table([
    ["Blood sugar(mg/dL)",str(inp.get("blood_sugar","—")),
    "Cholesterol(mg/dL)",str(inp.get("cholesterol","—"))],
    ["Systolic BP(mmHg)",str(inp.get("blood_pressure_systolic","—")),
    "Diabetes","Yes"if inp.get("diabetes")else"No"],
    ["Smoking","Yes"if inp.get("smoking")else"No",
    "Alcohol","Yes"if inp.get("alcohol")else"No"],
    ["Family history","Yes"if inp.get("family_history")else"No",
    "",""],
    ],col_widths=[40*mm,45*mm,40*mm,45*mm]))

#Symptoms

sym_keys=["numbness","tingling","burning_pain","muscle_weakness",
"loss_of_balance","foot_ulcers"]

selected=[k.replace("_","").title()for k in sym_keys if inp.get(k)]

story.append(Paragraph("Reported Symptoms",s["h2"]))

```

```

story.append(Paragraph(
    ", ".join(selected) if selected else "<i>No neurological symptoms
reported.</i>",
    s["body"]))

#Risk summary

story.append(Paragraph("Final AI Prediction", s["h2"]))

badge_color = _risk_color(level)

summary_tbl = Table([
    Paragraph(f"<font
color='{badge_color.hexval()}'><b>{level.upper()} RISK</b></font>",
    s["body"]),
    Paragraph(f"<b>Risk score:</b>{score}%", s["body"]),
    Paragraph(f"<b>Confidence:</b>{confidence if confidence is not None
else '—'}%",
    s["body"]),
    ], colWidths=[55*mm, 55*mm, 55*mm])

summary_tbl.setStyle(TableStyle([
    ("BACKGROUND", (0,0), (-1,-1), TEAL_LIGHT),
    ("BOX", (0,0), (-1,-1), 0.5, BORDER),
    ("TOPPADDING", (0,0), (-1,-1), 8),
    ("BOTTOMPADDING", (0,0), (-1,-1), 8),
    ("LEFTPADDING", (0,0), (-1,-1), 10),
    ]))

story.append(summary_tbl)

```

```

story.append(Spacer(1,6))

story.append(_risk_bar(score,level))


#Component breakdown

img_analyses=prediction.get("image_analyses")or{}

components={"Clinical(50%)":int(prediction.get("clinical_score",score))}

if"hand"in img_analyses:

components["Hand image(15%)"]=int(img_analyses["hand"]["score"])

if"foot"in img_analyses:

components["Foot image(15%)"]=int(img_analyses["foot"]["score"])

if"thermal_hand"in img_analyses or"thermal_foot"in img_analyses:

t_scores=[img_analyses[k]["score"]

for k in("thermal_hand","thermal_foot")if k in img_analyses]

components["Thermal(20%)"]=int(sum(t_scores)/len(t_scores))if t_scores

else 0

if len(components)>1:

story.append(Paragraph("Multimodal Component Scores",s["h2"]))

story.append(_component_chart(components))

#AI explanation—markdown-aware rendering with bullets and clean

headings

story.append(Paragraph("AI Clinical Explanation",s["h2"]))

explanation=prediction.get("ai_explanation","")or""

story.extend(render_explanation(explanation,s))

#Image analyses with thumbnails—include ORIGINAL,OVERLAY,and

```

THERMAL maps

if img_analyses:

story.append(PageBreak())

story.append(Paragraph("Image Analysis",s["h2"]))

for key,label in[

("hand","Hand Image(Standard)"),

("foot","Foot Image(Standard)"),

("thermal_hand","Thermal Hand Image"),

("thermal_foot","Thermal Foot Image"),

]:

a=img_analyses.get(key)

if not a:

continue

block=[]

block.append(Paragraph(f"{label}",s["body"]))

block.append(Paragraph(

f"AI

score:{a['score']} Confidence:{a['confidence'

]",

s["bodyMuted"]))

block.append(Spacer(1,4))

#Findings as bullet list

for finding in a.get("findings",[]):


```
tbl=Table([imgs_row,caps_row],colWidths=[col_w]*cols)

tbl.setStyle(TableStyle([

("ALIGN",(0,0),(-1,-1),"CENTER"),

("VALIGN",(0,0),(-1,0),"MIDDLE"),

("BOTTOMPADDING",(0,0),(-1,0),4),

("TOPPADDING",(0,1),(-1,1),2),

]))

block.append(tbl)

block.append(Spacer(1,12))

story.append(KeepTogether(block))


#Recommendations

recs=prediction.get("recommendations",[])

if recs:

story.append(Paragraph("Medical Recommendations",s["h2"]))

for r in recs:

story.append(Paragraph(f"•{r}",s["body"]))

story.append(Spacer(1,2))

story.append(Spacer(1,14))

story.append(Paragraph(

"<i>This report was generated by an AI-assisted decision support system."

"It is intended to supplement,not replace,professional medical judgment."

"Please consult a qualified neurologist for diagnosis and treatment

planning.</i>",
```

```
s["small"])))

doc.build(story,onFirstPage=_header_footer,onLaterPages=_header_footer
)

return buf.getvalue()
```

Fronted

Dashboard.jsx

```
import {useEffect,useState}from"react";

import {api}from"../lib/api";

import {motion}from"framer-motion";

import {useAuth}from"../context/AuthContext";

import {Link}from"react-router-dom";

import {Users,AlertTriangle,ShieldCheck,Activity,ArrowUpRight,Plus,Sparkles}from"lucide-react";

import {

  PieChart,Pie,Cell,ResponsiveContainer,Tooltip,

  BarChart,Bar,XAxis,YAxis,CartesianGrid,Legend,

}from"recharts";

const
```

```
RISK_COLORS={Low:"#10B981",Moderate:"#F59E0B",High:"#E11D4
```

```
8"};
```

```
const StatCard=({icon:Icon,label,value,sub,accent,testid,index})=>(  


```

```
  <motion.div
```

```
    initial={{opacity:0,y:14}}
```

```
    animate={{opacity:1,y:0}}
```

```
    transition={{duration:0.45,delay:index*0.08}}
```

```
    whileHover={{y:-2}}
```

```
    data-testid={testid}
```

```
    className="rounded-xl border border-border/60 bg-card p-5
```

```
    hover:shadow-md transition-all"
```

```
  >
```

```
  <div className="flex items-start justify-between mb-4">
```

```
    <div className={`h-10 w-10 rounded-lg flex items-center justify-  
center${accent}`}>
```

```
      <Icon className="h-5 w-5"strokeWidth={1.75}/>
```

```
    </div>
```

```
    <ArrowUpRight className="h-4 w-4 text-muted-foreground"/>
```

```
  </div>
```

```
  <p className="text-xs uppercase tracking-[0.12em]text-muted-  
foreground">{label}</p>
```

```
  <p className="text-3xl font-light tracking-tight mt-2">{value}</p>
```

```
  {sub}&&<p className="text-xs text-muted-foreground mt-  
2">{sub}</p>}
```

```

</motion.div>

);

export default function Dashboard(){

const {user}=useAuth();

const[stats,setStats]=useState(null);

const[loading,setLoading]=useState(true);

useEffect(()=>{

api.get("/stats")

.then((r)=>setStats(r.data))

.finally(()=>setLoading(false));

},[]);

if(loading){

return(

<div className="p-8 flex items-center justify-center min-h-

[60vh]"data-testid="dashboard-loading">

<div className="h-10 w-10 rounded-full border-2 border-primary

border-t-transparent animate-spin"/>

</div>

);

}

const empty=stats?.total_patients===0;

return(

<div className="p-6 lg:p-10 max-w-[1500px]mx-auto" data-

testid="dashboard-page">

```

```

{ /*Header*/}

<div className="flex flex-col md:flex-row md:items-end md:justify-
between gap-4 mb-10">

  <div>

    <h1 className="text-3xl sm:text-4xl font-light tracking-tight">

      Welcome back,<span className="font-medium text-
primary">{user?.full_name?.split("")[0]}</span>

    </h1>

  </div>

  <Link

    to="/new"

    data-testid="dashboard-new-prediction-button"

    className="inline-flex items-center gap-2 rounded-md bg-primary px-
4 py-2.5 text-sm font-medium text-primary-foreground hover:bg-
primary/90 transition-colors self-start md:self-auto"

    >

    <Plus className="h-4 w-4"/>New Prediction

  </Link>

</div>

{ /*Stat cards*/}

<div className="grid grid-cols-2 lg:grid-cols-4 gap-4 mb-8">

  <StatCard testid="stat-total"index={0}icon={Users}label="Total
Patients"value={stats.total_patients}

```

```

sub="Lifetime assessments"accent="bg-primary/10 text-primary"/>

<StatCard testid="stat-

high"index={1}icon={AlertTriangle}label="High

Risk"value={stats.high_risk}

sub={stats.total_patients?`${Math.round(stats.high_risk*100/stats.total_

patients)}}%of cases`:"—"}

accent="bg-rose-500/10 text-rose-500"/>

<StatCard testid="stat-

moderate"index={2}icon={Activity}label="Moderate

Risk"value={stats.moderate_risk}

sub={stats.total_patients?`${Math.round(stats.moderate_risk*100/stats.t

otal_patients)}}%of cases`:"—"}

accent="bg-amber-500/10 text-amber-500"/>

<StatCard testid="stat-low"index={3}icon={ShieldCheck}label="Low

Risk"value={stats.low_risk}

sub={`Avg score:${stats.average_score}`}accent="bg-emerald-500/10

text-emerald-500"/>

</div>

{empty?(

<motion.div

initial={{opacity:0}}animate={{opacity:1}}

data-testid="dashboard-empty-state"

className="rounded-xl border border-dashed border-border bg-card/40

p-12 text-center"

```

>

<div className="h-14 w-14 mx-auto rounded-xl bg-primary/10 flex

items-center justify-center mb-4">

<Sparkles className="h-6 w-6 text-primary"/>

</div>

<h3 className="text-xl font-medium mb-2">No assessments yet</h3>

<p className="text-sm text-muted-foreground max-w-md mx-auto mb-

6">

Run your first AI-powered neuropathy prediction to populate your

dashboard with real patient analytics.

</p>

<Link to="/new" data-testid="empty-create-button"

className="inline-flex items-center gap-2 rounded-md bg-primary px-

5 py-2.5 text-sm font-medium text-primary-foreground hover:bg-

primary/90">

<Plus className="h-4 w-4"/>Create your first prediction

</Link>

</motion.div>

):(

<div className="grid grid-cols-1 lg:grid-cols-12 gap-6">

{/*Monthly trend*/}

<motion.div

initial={{ opacity:0,y:12}}animate={{ opacity:1,y:0}}transition={{ delay

:0.35}}

```

className="lg:col-span-8 rounded-xl border border-border/60 bg-card
p-6"
data-testid="chart-monthly"
>
<div className="flex items-start justify-between mb-6">
<div>
<p className="text-xs uppercase tracking-[0.12em]text-muted-
foreground">Monthly Trend</p>
<h3 className="text-lg font-medium mt-1">Assessments by risk
level</h3>
</div>
</div>
<ResponsiveContainer width="100%"height={280}>
<BarChart data={stats.monthly_trend}>
<CartesianGrid stroke="hsl(var(--border))"strokeDasharray="3
3"vertical={false}/>
<XAxis dataKey="month"stroke="hsl(var(--muted-
foreground))"fontSize={11}/>
<YAxis stroke="hsl(var(--muted-foreground))"fontSize={11}/>
<Tooltip contentStyle={{
background:"hsl(var(--card))",border:"1px solid hsl(var(--border))",
borderRadius:8,fontSize:12,
}}/>
<Legend wrapperStyle={{fontSize:12}}/>

```

```

<Bar
dataKey="low"stackId="a"fill={RISK_COLORS.Low}radius={[0,0,0,0]}/
>

<Bar
dataKey="moderate"stackId="a"fill={RISK_COLORS.Moderate}/>

<Bar
dataKey="high"stackId="a"fill={RISK_COLORS.High}radius={[4,4,0,0]
}/>

</BarChart>

</ResponsiveContainer>

</motion.div>

{/*Risk distribution*/}

<motion.div
initial={{opacity:0,y:12}}animate={{opacity:1,y:0}}transition={{delay
:0.45}}

className="lg:col-span-4 rounded-xl border border-border/60 bg-card
p-6"

data-testid="chart-risk-distribution"

>

<p className="text-xs uppercase tracking-[0.12em]text-muted-
foreground">Risk Distribution</p>

<h3 className="text-lg font-medium mt-1 mb-4">All patients</h3>

<ResponsiveContainer width="100%"height={220}>

<PieChart>

```

```

<Pie data={stats.risk_distribution}dataKey="value"nameKey="name"
innerRadius={55}outerRadius={85}paddingAngle={3}>

{stats.risk_distribution.map((e)=>(
  <Cell key={e.name}fill={RISK_COLORS[e.name]}/>
))}

</Pie>

<Tooltip contentType={{
  background:"hsl(var(--card))",border:"1px solid hsl(var(--border))",
  borderRadius:8,fontSize:12,
}}/>

</PieChart>

</ResponsiveContainer>

<div className="space-y-2 mt-2">

  {stats.risk_distribution.map((e)=>(
    <div key={e.name}className="flex items-center justify-between text-
sm">

      <div className="flex items-center gap-2">

        <span className="h-2.5 w-2.5 rounded-
full"style={{background:RISK_COLORS[e.name]}}/>

        <span>{e.name}</span>

      </div>

        <span className="font-medium">{e.value}</span>

      </div>

    ))}

```

```

</div>

</motion.div>

{/*Recent activity*/}

<motion.div

initial={{opacity:0,y:12}}animate={{opacity:1,y:0}}transition={{delay

:0.55}}

className="lg:col-span-8 rounded-xl border border-border/60 bg-card

p-6"

data-testid="recent-activity"

>

<div className="flex items-center justify-between mb-4">

<div>

<p className="text-xs uppercase tracking-[0.12em]text-muted-

foreground">Recent Activity</p>

<h3 className="text-lg font-medium mt-1">Latest assessments</h3>

</div>

<Link to="/patients"className="text-xs text-primary

hover:underline"data-testid="view-all-link">

View all→

</Link>

</div>

<div className="divide-y divide-border/60">

{stats.recent_activity.map((r)=>(

```

```

<Link
key={r.id}to={`patients/${r.id}`}
data-testid={`activity-row-${r.id}`}
className="flex items-center justify-between py-3 hover:bg-muted/40-
mx-2 px-2 rounded transition-colors"
>

<div className="flex items-center gap-3 min-w-0">
  <div className="h-9 w-9 rounded-full bg-primary/10 text-primary flex
items-center justify-center text-xs font-semibold flex-shrink-0">
    {r.patient_name.split("").map((n)=>n[0]).slice(0,2).join("").toUpperCas
e()}
  </div>
  <div className="min-w-0">
    <p className="font-medium text-sm truncate">{r.patient_name}</p>
    <p className="text-xs text-muted-foreground">
      {new Date(r.created_at).toLocaleString()}
    </p>
  </div>
</div>

<div className="flex items-center gap-3">
  <span className="text-sm font-medium tabular-
nums">{r.risk_score}</span>
  <span
className="px-2 py-0.5 rounded-full text-[10px]font-semibold

```

tracking-wide uppercase"

```

    style={{
      background:`${RISK_COLORS[r.risk_level]}22`,
      color:RISK_COLORS[r.risk_level],
    }}
  >

  {r.risk_level}

</span>

</div>

</Link>

)))}

</div>

</motion.div>

{ /*Age distribution*/}

<motion.div

  initial={{opacity:0,y:12}}animate={{opacity:1,y:0}}transition={{delay

:0.65}}

  className="lg:col-span-4 rounded-xl border border-border/60 bg-card

p-6"

  data-testid="chart-age"

  >

  <p className="text-xs uppercase tracking-[0.12em]text-muted-

foreground">Age Distribution</p>

  <h3 className="text-lg font-medium mt-1 mb-4">Patients by

```

age</h3>

```
<ResponsiveContainer width="100%"height={220}>
```

```
<BarChart data={stats.age_distribution}layout="vertical">
```

```
<CartesianGrid stroke="hsl(var(--border))"strokeDasharray="3
```

```
3"horizontal={false}/>
```

```
<XAxis type="number"stroke="hsl(var(--muted-
```

```
foreground))"fontSize={11}/>
```

```
<YAxis type="category"dataKey="bucket"stroke="hsl(var(--muted-
```

```
foreground))"fontSize={11}width={50}/>
```

```
<Tooltip contentStyle={{
```

```
background:"hsl(var(--card))",border:"1px solid hsl(var(--border))",
```

```
borderRadius:8,fontSize:12,
```

```
}}/>
```

```
<Bar dataKey="count"fill="hsl(var(--primary))"radius={[0,4,4,0]}/>
```

```
</BarChart>
```

```
</ResponsiveContainer>
```

```
</motion.div>
```

```
</div>
```

```
)}
```

```
</div>
```

```
);
```

```
}
```

Login.jsx

```
import {useState} from "react";

import {Link,useNavigate,Navigate} from "react-router-dom";

import {useAuth} from "../context/AuthContext";

import {Activity,Loader2} from "lucide-react";

import {toast} from "sonner";

import {motion} from "framer-motion";

export default function Login(){

  const {user,login}=useAuth();

  const navigate=useNavigate();

  const [email,setEmail]=useState("");

  const [password,setPassword]=useState("");

  const [loading,setLoading]=useState(false);

  if(user)return<Navigate to="/"replace/>;

  const submit=async(e)=>{

    e.preventDefault();

    setLoading(true);

    try{

      await login(email,password);

      toast.success("Welcome back!");

      navigate("/");

    }catch(err){
```

```

toast.error(err?.response?.data?.detail||"Login failed");

}finally{

setLoading(false);

}

};

return(

<div className="min-h-screen grid lg:grid-cols-2" data-testid="login-
page">

  { /*Left form*/ }

  <div className="flex flex-col justify-center px-6 sm:px-12 lg:px-20
py-12 bg-background">

    <motion.div

      initial={{ opacity:0,y:12 }}

      animate={{ opacity:1,y:0 }}

      transition={{ duration:0.5 }}

      className="max-w-md w-full mx-auto"

    >

      <div className="flex items-center gap-2 mb-10">

        <div className="h-10 w-10 rounded-xl bg-primary/10 flex items-
center justify-center">

          <Activity className="h-5 w-5 text-primary" strokeWidth={1.75}/>

        </div>

        <div>

          <p className="font-semibold tracking-tight">NeuroDetect</p>

```

```
<p className="text-[10px]uppercase tracking-[0.18em]text-muted-foreground">
  Healthcare
</p>
</div>
</div>

<p className="text-xs uppercase tracking-[0.18em]text-muted-foreground mb-3">
  Welcome back
</p>

<h1 className="text-4xl sm:text-5xl font-light tracking-tight mb-8">

  Sign in to your<span className="text-primary font-medium">workspace</span>

</h1>

<form onSubmit={submit}className="space-y-5">

  <div>

    <label className="text-xs uppercase tracking-[0.1em]text-muted-foreground">Email</label>

    <input
      type="email"
      required
      value={email}
      onChange={(e)=>setEmail(e.target.value)}
      data-testid="login-email-input"
```

```

className="mt-2 w-full rounded-md border border-input bg-
background px-3 py-2.5 text-sm focus:outline-none focus:ring-2
focus:ring-primary/30 focus:border-primary transition-all"

placeholder="doctor@hospital.com"

/>

</div>

<div>

<label className="text-xs uppercase tracking-[0.1em]text-muted-
foreground">Password</label>

<input

type="password"

required

value={password}

onChange={(e)=>setPassword(e.target.value)}

data-testid="login-password-input"

className="mt-2 w-full rounded-md border border-input bg-
background px-3 py-2.5 text-sm focus:outline-none focus:ring-2
focus:ring-primary/30 focus:border-primary transition-all"

placeholder="....."

/>

</div>

<button

type="submit"

disabled={loading}

```

```

data-testid="login-submit-button"

className="w-full inline-flex items-center justify-center gap-2
rounded-md bg-primary px-4 py-2.5 text-sm font-medium text-primary-
foreground hover:bg-primary/90 transition-colors disabled:opacity-60"
>

{loading}&&<Loader2 className="h-4 w-4 animate-spin"/>}

Sign In

</button>

</form>

<p className="mt-6 text-sm text-muted-foreground">

New here?{""}

<Link to="/register"className="text-primary font-medium
hover:underline"data-testid="link-to-register">

Create an account

</Link>

</p>

</motion.div>

</div>

{/*Right decorative panel*/}

<div className="hidden lg:block relative bg-primary overflow-
hidden">

<div className="absolute inset-0 grain opacity-30"/>

<div className="absolute inset-0"

style={{

```

```

background:"radial-gradient(circle at
30%20%,rgba(255,255,255,0.18),transparent 60%),radial-gradient(circle
at 70%80%,rgba(20,184,166,0.5),transparent 50%)",

}}/>

<svg className="absolute inset-x-0 top-1/2-translate-y-1/2 w-full h-40
opacity-60"

viewBox="0 0 800 100"preserveAspectRatio="none"fill="none">

<path d="M0 50 L100 50 L120 30 L140 70 L160 20 L180 80 L200 50
L320 50 L340 35 L360 65 L380 25 L400 75 L420 50 L600 50 L620 40
L640 60 L660 50 L800 50"

stroke="white"strokeWidth="2"className="ecg-line"/>

</svg>

</div>

</div>

);

}

```

NewPrediction.jsx

```

import {useState} from "react";

import {useNavigate} from "react-router-dom";

import {api} from "../lib/api";

import {motion, AnimatePresence} from "framer-motion";

import {toast} from "sonner";

import {Loader2, Sparkles, User, HeartPulse, Activity, FileText, Camera, Thermometer, Scan} from "lucide-react";

```

```
import ImageUploader from"../components/ImageUploader";

const symptomFields=[

  {key:"numbness",label:"Numbness"},

  {key:"tingling",label:"Tingling"},

  {key:"burning_pain",label:"Burning pain"},

  {key:"muscle_weakness",label:"Muscle weakness"},

  {key:"loss_of_balance",label:"Loss of balance"},

  {key:"foot_ulcers",label:"Foot ulcers"},

];

const lifestyleFields=[

  {key:"diabetes",label:"Diabetes"},

  {key:"smoking",label:"Smoking"},

  {key:"alcohol",label:"Alcohol consumption"},

  {key:"family_history",label:"Family history"},

];

const defaultForm={

  patient_name:"",age:45,gender:"Male",bmi:24.5,

  blood_sugar:110,cholesterol:190,blood_pressure_systolic:125,

  physical_activity:"Moderate",

  diabetes:false,smoking:false,alcohol:false,family_history:false,

  numbness:false,tingling:false,burning_pain:false,

  muscle_weakness:false,loss_of_balance:false,foot_ulcers:false,

};
```

```
const Section=({icon:Icon,title,subtitle,span="md:col-span-
12",children,testid})=>(
  <div className={`rounded-xl border border-border/60 bg-card p-
6${span}`} data-testid={testid}>
    <div className="flex items-center gap-3 mb-5">
      <div className="h-9 w-9 rounded-lg bg-primary/10 flex items-center
justify-center">
        <Icon className="h-4 w-4 text-primary" strokeWidth={1.75}/>
      </div>
      <div>
        <h3 className="font-medium">{title}</h3>
        {subtitle}&&<p className="text-xs text-muted-
foreground">{subtitle}</p>}
      </div>
    </div>
  </div>
  <div className="grid grid-cols-1 sm:grid-cols-2 gap-
4">{children}</div>
</div>
);
const Field=({label,children})=>(
  <label className="block">
    <span className="text-xs uppercase tracking-[0.1em]text-muted-
foreground">{label}</span>
    <div className="mt-2">{children}</div>
```

```

</label>

);

const inputCls=

"w-full rounded-md border border-input bg-background px-3 py-2 text-
sm focus:outline-none focus:ring-2 focus:ring-primary/30 focus:border-
primary transition-all";

const Toggle=({checked,onChange,label,testid})=>{

<button

type="button"onClick={()=>onChange(!checked)}data-testid={testid}

className={`flex items-center justify-between rounded-md border px-3
py-2.5 text-sm transition-all${

checked?"border-primary bg-primary/5 text-foreground":"border-input
bg-background text-muted-foreground hover:border-primary/40"

}}`

>

<span>{label}</span>

<span className={`h-4 w-7 rounded-full p-0.5 transition-
colors${checked?"bg-primary":"bg-muted"}}`>

<span className={`block h-3 w-3 rounded-full bg-white transition-
transform${checked?"translate-x-3":"":""}`>/>

</span>

</button>

);

```

```

export default function NewPrediction(){

const[form,setForm]=useState(defaultForm);

const[images,setImages]=useState({

hand:null,foot:null,thermal_hand:null,thermal_foot:null,

});

const[loading,setLoading]=useState(false);

const[progress,setProgress]=useState(0);

const[stage,setStage]=useState("");

const navigate=useNavigate();

const set=(k)=>(v)=>setForm((f)=>({...f,[k]:v}));

const setImg=(k)=>(f)=>setImages((s)=>({...s,[k]:f}));

const submit=async(e)=>{

e.preventDefault();

if(!form.patient_name.trim()){

toast.error("Patient name is required");

return;

}

setLoading(true);setProgress(8);setStage("Uploading clinical data...");

try{

const fd=new FormData();

fd.append("data",JSON.stringify(form));

if(images.hand)fd.append("hand_image",images.hand);

if(images.foot)fd.append("foot_image",images.foot);

if(images.thermal_hand)fd.append("thermal_hand_image",images.therm

```

```

al_hand);

if(images.thermal_foot)fd.append("thermal_foot_image",images.therma
l_foot);

//Simulated multi-stage progress(real upload progress+AI inference)

const stages=[

[25,"Preprocessing images with OpenCV..."],

[45,"Running AI image analysis..."],

[70,"Generating thermal visualizations..."],

[88,"Running Claude AI explanation..."],

];

let idx=0;

const interval=setInterval(=>{

if(idx<stages.length){

setProgress(stages[idx][0]);setStage(stages[idx][1]);idx++;

}

},1200);

const{data}=await api.post("/predictions",fd,{

headers:{"Content-Type":"multipart/form-data"},

onUploadProgress:(evt)=>{

if(evt.total)setProgress(Math.max(10,Math.min(22,(evt.loaded/evt.total)

*22)));

},

});

clearInterval(interval);

```

```

setProgress(100);setStage("Complete");

toast.success("Multimodal prediction complete");

navigate(`/patients/${data.id}`);

}catch(err){

toast.error(err?.response?.data?.detail||"Prediction failed");

}finally{

setLoading(false);

}

};

const imageCount=Object.values(images).filter(Boolean).length;

return(

<div className="p-6 lg:p-10 max-w-[1500px]mx-auto" data-
testid="new-prediction-page">

<div className="mb-8">

<h1 className="text-3xl sm:text-4xl font-light tracking-tight">

New<span className="font-medium text-
primary">Neuropathy</span>Prediction

</h1>

</div>

<motion.form

onSubmit={submit}

initial={{opacity:0,y:10}}animate={{opacity:1,y:0}}transition={{durat
ion:0.4}}

className="grid grid-cols-1 md:grid-cols-12 gap-6"

```

```

>

<Section testid="section-
demographics"icon={User}title="Demographics"subtitle="Basic patient
information"span="md:col-span-6">

  <Field label="Patient name">

    <input type="text"required value={form.patient_name}
    onChange={(e)=>set("patient_name")(e.target.value)}
    data-testid="patient-name-
input"classname={inputCls}placeholder="Full name"/>

  </Field>

  <Field label="Age">

    <input type="number"min="1"max="120"required value={form.age}
    onChange={(e)=>set("age")(parseInt(e.target.value)||0)}
    data-testid="patient-age-input"classname={inputCls}/>

  </Field>

  <Field label="Gender">

    <select
value={form.gender}onChange={(e)=>set("gender")(e.target.value)}
    data-testid="patient-gender-select"classname={inputCls}>

      <option>Male</option><option>Female</option><option>Other</opti
on>

    </select>

  </Field>

  <Field label="BMI(kg/m²)">

```

```

<input type="number"step="0.1"min="10"max="60"required
value={ form.bmi}

onChange={(e)=>set("bmi")(parseFloat(e.target.value)||0)}

data-testid="patient-bmi-input"className={ inputCls}/>

</Field>

</Section>

<Section testid="section-
vitals"icon={HeartPulse}title="Vitals&Labs"subtitle="Recent
measurements"span="md:col-span-6">

<Field label="Blood sugar(mg/dL)">

<input type="number"min="40"max="600"required
value={ form.blood_sugar}

onChange={(e)=>set("blood_sugar")(parseFloat(e.target.value)||0)}

data-testid="patient-bloodsugar-input"className={ inputCls}/>

</Field>

<Field label="Cholesterol(mg/dL)">

<input type="number"min="80"max="400"required
value={ form.cholesterol}

onChange={(e)=>set("cholesterol")(parseFloat(e.target.value)||0)}

data-testid="patient-cholesterol-input"className={ inputCls}/>

</Field>

<Field label="Systolic blood pressure(mmHg)">

<input type="number"min="70"max="220"required
value={ form.blood_pressure_systolic}

```

```

onChange={{(e)=>set("blood_pressure_systolic")(parseInt(e.target.value
)||0)}}

data-testid="patient-bp-input"className={inputCls}/>

</Field>

<Field label="Physical activity">

<select

value={form.physical_activity}onChange={{(e)=>set("physical_activity")(
e.target.value)}}

data-testid="patient-activity-select"className={inputCls}>

<option>Low</option><option>Moderate</option><option>High</opti
on>

</select>

</Field>

</Section>

<Section testid="section-
lifestyle"icon={Activity}title="Medical&Lifestyle"subtitle="Tap to
toggle"span="md:col-span-6">

{lifestyleFields.map((f)=>(

<Toggle key={f.key}label={f.label}

checked={form[f.key]}onChange={set(f.key)}

testid={`toggle-${f.key}`}>

)))}

</Section>

<Section testid="section-symptoms"icon={FileText}title="Neurological

```

Symptoms"subtitle="Tap to toggle observed symptoms"span="md:col-

span-6">

```
{symptomFields.map((f)=>{
```

```
<Toggle key={f.key}label={f.label}
```

```
checked={form[f.key]}onChange={set(f.key)}
```

```
testid={`toggle-${f.key}`}>
```

```
}}}
```

```
</Section>
```

```
{/*Standard images*/}
```

```
<div className="md:col-span-6 rounded-xl border border-border/60
```

```
bg-card p-6"data-testid="section-standard-images">
```

```
<div className="flex items-center gap-3 mb-5">
```

```
<div className="h-9 w-9 rounded-lg bg-primary/10 flex items-center
```

```
justify-center">
```

```
<Camera className="h-4 w-4 text-primary"strokeWidth={1.75}/>
```

```
</div>
```

```
<div>
```

```
<h3 className="font-medium">Standard Images</h3>
```

```
<p className="text-xs text-muted-foreground">Phone/camera
```

```
photos.AI generates pseudo-thermal visualization.</p>
```

```
</div>
```

```
</div>
```

```
<div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
```

```
<ImageUploader label="Hand image"testid="hand"
```

```

file={images.hand}onChange={setImg("hand")}hint="Palm or back of
hand"/>

<ImageUploader label="Foot image"testid="foot"

file={images.foot}onChange={setImg("foot")}hint="Sole or top of
foot"/>

</div>

</div>

{ /*Thermal images*/}

<div className="md:col-span-6 rounded-xl border border-border/60
bg-card p-6" data-testid="section-thermal-images">

<div className="flex items-center gap-3 mb-5">

<div className="h-9 w-9 rounded-lg bg-amber-500/10 flex items-
center justify-center">

<Thermometer className="h-4 w-4 text-amber-
500"strokeWidth={1.75}/>

</div>

<div>

<h3 className="font-medium">Thermal Images</h3>

<p className="text-xs text-muted-foreground">Real FLIR/thermal
camera images for hotspot&asymmetry analysis.</p>

</div>

</div>

<div className="grid grid-cols-1 sm:grid-cols-2 gap-4">

<ImageUploader label="Thermal hand"testid="thermal-

```

```

hand"accent="thermal"

file={images.thermal_hand}onChange={setImg("thermal_hand")}hint=
"Real thermal image"/>

<ImageUploader label="Thermal foot"testid="thermal-
foot"accent="thermal"

file={images.thermal_foot}onChange={setImg("thermal_foot")}hint="
Real thermal image"/>

</div>

</div>

<div className="md:col-span-12 flex flex-col-reverse md:flex-row
items-stretch md:items-center justify-end gap-3 pt-2">

{imageCount}>0&&(

<p className="text-xs text-muted-foreground mr-auto" data-
testid="image-count">

<Scan className="inline h-3.5 w-3.5 mr-1 text-primary"/>

{imageCount} image{imageCount>1?"s":""} attached—multimodal
analysis enabled

</p>

)}

<button

type="button"onClick={()=>{setForm(defaultForm);setImages({hand:null
,foot:null,thermal_hand:null,thermal_foot:null});}}

data-testid="reset-form-button"

className="rounded-md border border-input bg-background px-5 py-

```

2.5 text-sm font-medium hover:bg-muted transition-colors">

Reset

</button>

<button type="submit" disabled={loading}

data-testid="submit-prediction-button"

className="inline-flex items-center justify-center gap-2 rounded-md

bg-primary px-6 py-2.5 text-sm font-medium text-primary-foreground

hover:bg-primary/90 transition-colors disabled:opacity-60">

{loading?<Loader2 className="h-4 w-4 animate-spin"/>:<Sparkles

className="h-4 w-4"/>}

{loading?"Analyzing... ":"Run Multimodal AI Prediction"}

</button>

</div>

</motion.form>

{/*AI scanning overlay*/}

<AnimatePresence>

{loading&&(

<motion.div

initial={{opacity:0}}animate={{opacity:1}}exit={{opacity:0}}

className="fixed inset-0 z-50 bg-background/85 backdrop-blur-md

flex items-center justify-center p-6"

data-testid="ai-scanning-overlay"

>

<div className="max-w-md w-full text-center">

```

<div className="relative h-32 w-32 mx-auto mb-6">

  <motion.div className="absolute inset-0 rounded-full border-2 border-
primary/30"/>

  <motion.div className="absolute inset-2 rounded-full border-2 border-
primary/50"

    animate={{ rotate: 360 }} transition={{ duration: 3, repeat: Infinity, ease: "lin
ear" }} />

  <motion.div className="absolute inset-4 rounded-full border-2 border-
t-primary border-r-transparent border-b-transparent border-l-transparent"

    animate={{ rotate: -
360 }} transition={{ duration: 1.5, repeat: Infinity, ease: "linear" }} />

  <div className="absolute inset-0 flex items-center justify-center">

    <Scan className="h-10 w-10 text-primary"/>

  </div>

</div>

<p className="text-xs uppercase tracking-[0.18em] text-muted-
foreground mb-2">AI Scanning</p>

<h3 className="text-2xl font-light tracking-tight mb-3">Multimodal
Analysis</h3>

<p className="text-sm text-muted-foreground mb-6">{stage}</p>

<div className="h-1.5 rounded-full bg-muted overflow-hidden">

  <motion.div

    className="h-full bg-primary"

    animate={{ width: `${progress}%` }}

```

```

transition={{duration:0.4}}

/>

</div>

<p className="text-xs text-muted-foreground mt-2">{progress}%</p>

</div>

</motion.div>

)}

</AnimatePresence>

</div>

);

}

```

register.jsx

```

import {useState} from "react";

import {Link,useNavigate,Navigate} from "react-router-dom";

import {useAuth} from "../context/AuthContext";

import {Activity,Loader2} from "lucide-react";

import {toast} from "sonner";

import {motion} from "framer-motion";

export default function Register(){

const {user,register}=useAuth();

const navigate=useNavigate();

```

```
const[full_name,setFullName]=useState("");

const[email,setEmail]=useState("");

const[password,setPassword]=useState("");

const[loading,setLoading]=useState(false);


if(user)return<Navigate to="/"replace/>;

const submit=async(e)=>{

e.preventDefault();

setLoading(true);

try{

await register(full_name,email,password);

toast.success("Account created");

navigate("/");

}catch(err){

toast.error(err?.response?.data?.detail||"Registration failed");

}finally{

setLoading(false);

}

};

return(

<div className="min-h-screen grid lg:grid-cols-2" data-
```

```

testid="register-page">

  <div className="flex flex-col justify-center px-6 sm:px-12
lg:px-20 py-12 bg-background">

    <motion.div
initial={{opacity:0,y:12}}animate={{opacity:1,y:0}}transition
={{duration:0.5}}className="max-w-md w-full mx-auto">

      <div className="flex items-center gap-2 mb-10">

        <div className="h-10 w-10 rounded-xl bg-primary/10 flex
items-center justify-center">

          <Activity className="h-5 w-5 text-
primary"strokeWidth={1.75}/>

        </div>

        <div>

          <p className="font-semibold tracking-
tight">NeuroDetect</p>

          <p className="text-[10px]uppercase tracking-[0.18em]text-
muted-foreground">Healthcare</p>

        </div>

      </div>

    <p className="text-xs uppercase tracking-[0.18em]text-
muted-foreground mb-3">Get started</p>

```

```
<h1 className="text-4xl sm:text-5xl font-light tracking-tight
mb-8">
```

```
Create your<span className="text-primary font-
medium">account</span>
```

```
</h1>
```

```
<form onSubmit={submit}className="space-y-5">
```

```
<div>
```

```
<label className="text-xs uppercase tracking-[0.1em]text-
muted-foreground">Full name</label>
```

```
<input
```

```
type="text"required
```

```
value={full_name}onChange={(e)=>setFullName(e.target.valu
e)}
```

```
data-testid="register-name-input"
```

```
className="mt-2 w-full rounded-md border border-input bg-
background px-3 py-2.5 text-sm focus:outline-none focus:ring-2
focus:ring-primary/30 focus:border-primary transition-all"
```

```
placeholder="Dr.Aisha Khan"
```

```
/>
```

```
</div>
```

```
<div>
```

```

<label className="text-xs uppercase tracking-[0.1em]text-
muted-foreground">Email</label>

<input

type="email"required

value={email}onChange={(e)=>setEmail(e.target.value)}

data-testid="register-email-input"

className="mt-2 w-full rounded-md border border-input bg-
background px-3 py-2.5 text-sm focus:outline-none focus:ring-2
focus:ring-primary/30 focus:border-primary transition-all"

placeholder="you@clinic.com"

/>

</div>

<div>

<label className="text-xs uppercase tracking-[0.1em]text-
muted-foreground">Password</label>

<input

type="password"required

minLength={6}value={password}onChange={(e)=>setPasswor
d(e.target.value)}

data-testid="register-password-input"

className="mt-2 w-full rounded-md border border-input bg-

```

background px-3 py-2.5 text-sm focus:outline-none focus:ring-2

focus:ring-primary/30 focus:border-primary transition-all"

placeholder="Min.6 characters"

/>

</div>

<button type="submit" disabled={loading} data-

testid="register-submit-button"

className="w-full inline-flex items-center justify-center

gap-2 rounded-md bg-primary px-4 py-2.5 text-sm font-

medium text-primary-foreground hover:bg-primary/90

transition-colors disabled:opacity-60">

{loading}&&<Loader2 className="h-4 w-4 animate-spin"/>}

Create Account

</button>

</form>

<p className="mt-6 text-sm text-muted-foreground">

Already have an account?{""}

<Link to="/login" className="text-primary font-medium

hover:underline" data-testid="link-to-login">

Sign in

</Link>

```

</p>

</motion.div>

</div>

<div className="hidden lg:block relative bg-primary
overflow-hidden">

  <div className="absolute inset-0 grain opacity-30"/>

  <div className="absolute inset-0"
  style={{
    background:"radial-gradient(circle at
30%20%,rgba(255,255,255,0.18),transparent 60%),radial-
gradient(circle at 70%80%,rgba(20,184,166,0.5),transparent
50%)",
  }}/>

  <svg className="absolute inset-x-0 top-1/2-translate-y-1/2
w-full h-40 opacity-60"
  viewBox="0 0 800
100"preserveAspectRatio="none"fill="none">

    <path d="M0 50 L100 50 L120 30 L140 70 L160 20 L180 80
L200 50 L320 50 L340 35 L360 65 L380 25 L400 75 L420 50
L600 50 L620 40 L640 60 L660 50 L800 50"
stroke="white"strokeWidth="2"className="ecg-line"/>

```

</svg>

</div>

</div>

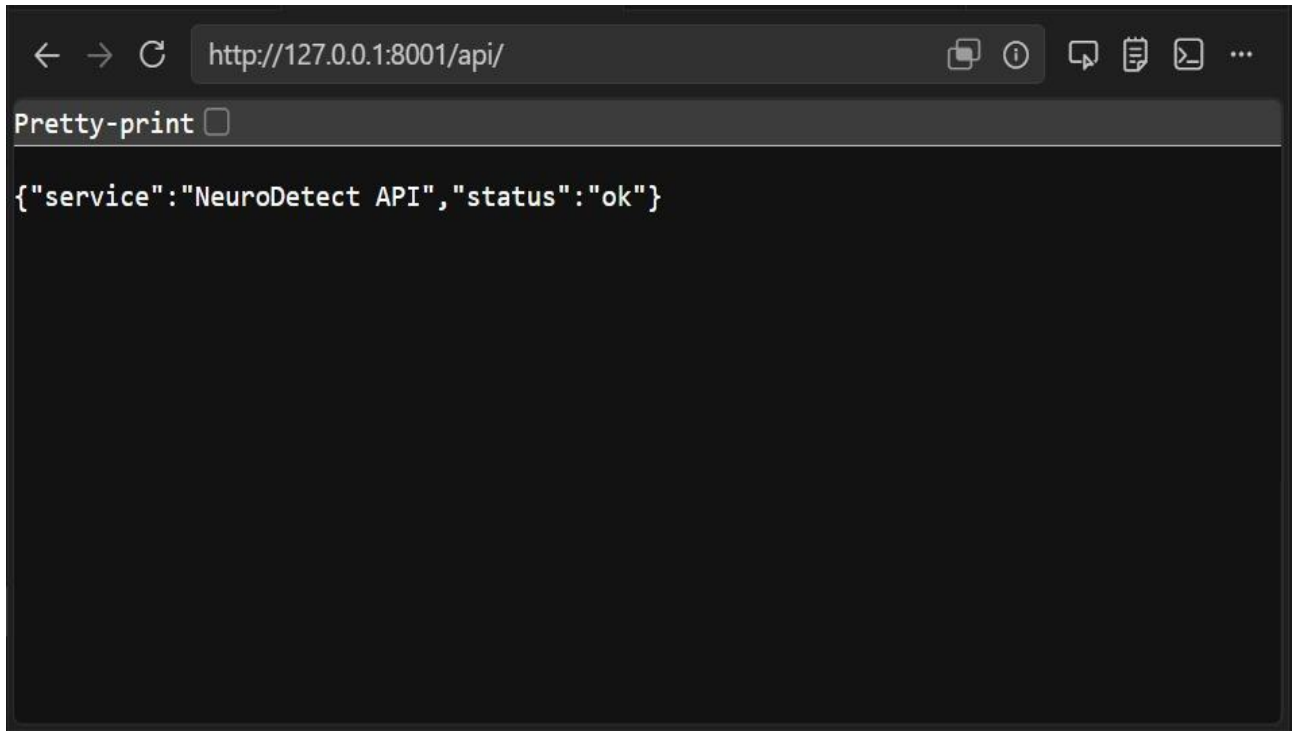
);

}

CHAPTER 8

RESULT

1. Backend Running



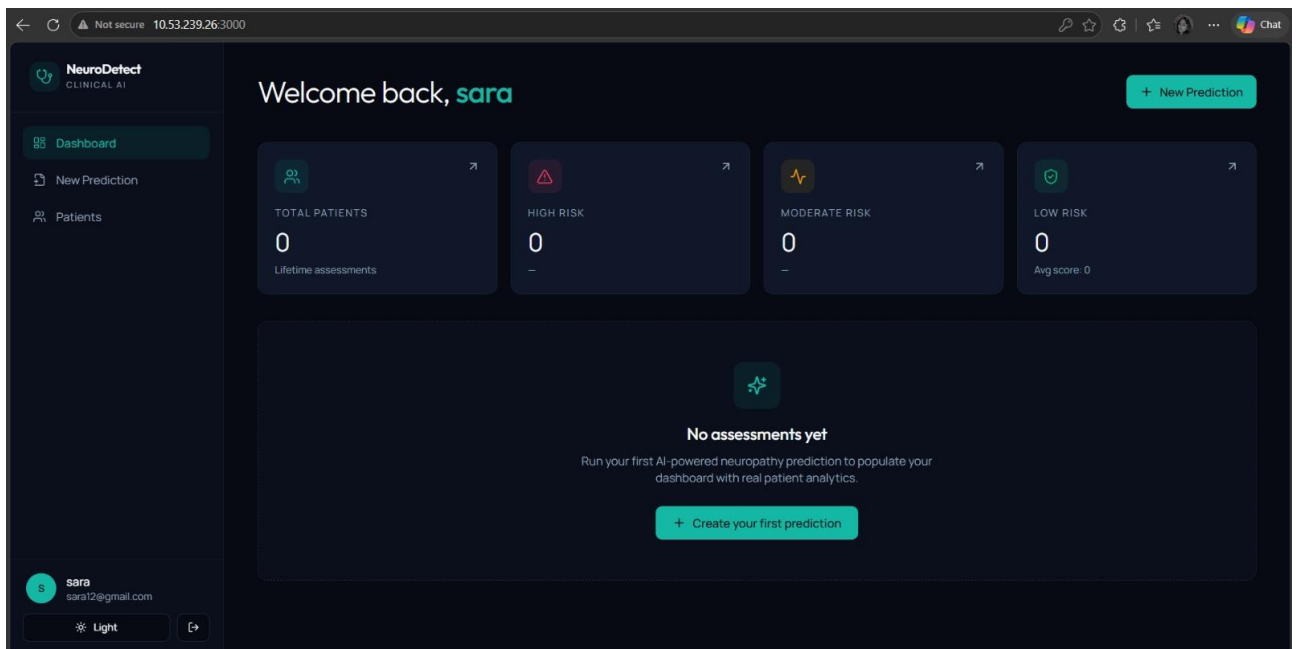
2. Login Page

The screenshot shows a web browser window with the URL `10.53.239.26:3000/register`. The page is for "NeuroDetect HEALTHCARE" and is titled "GET STARTED Create your account". It features a registration form with the following fields:

- FULL NAME:
- EMAIL:
- PASSWORD:

Below the password field is a "Create Account" button. At the bottom, there is a link: "Already have an account? [Sign in](#)". The right side of the page has a teal background with a white ECG line graphic.

3. Dashboard Page



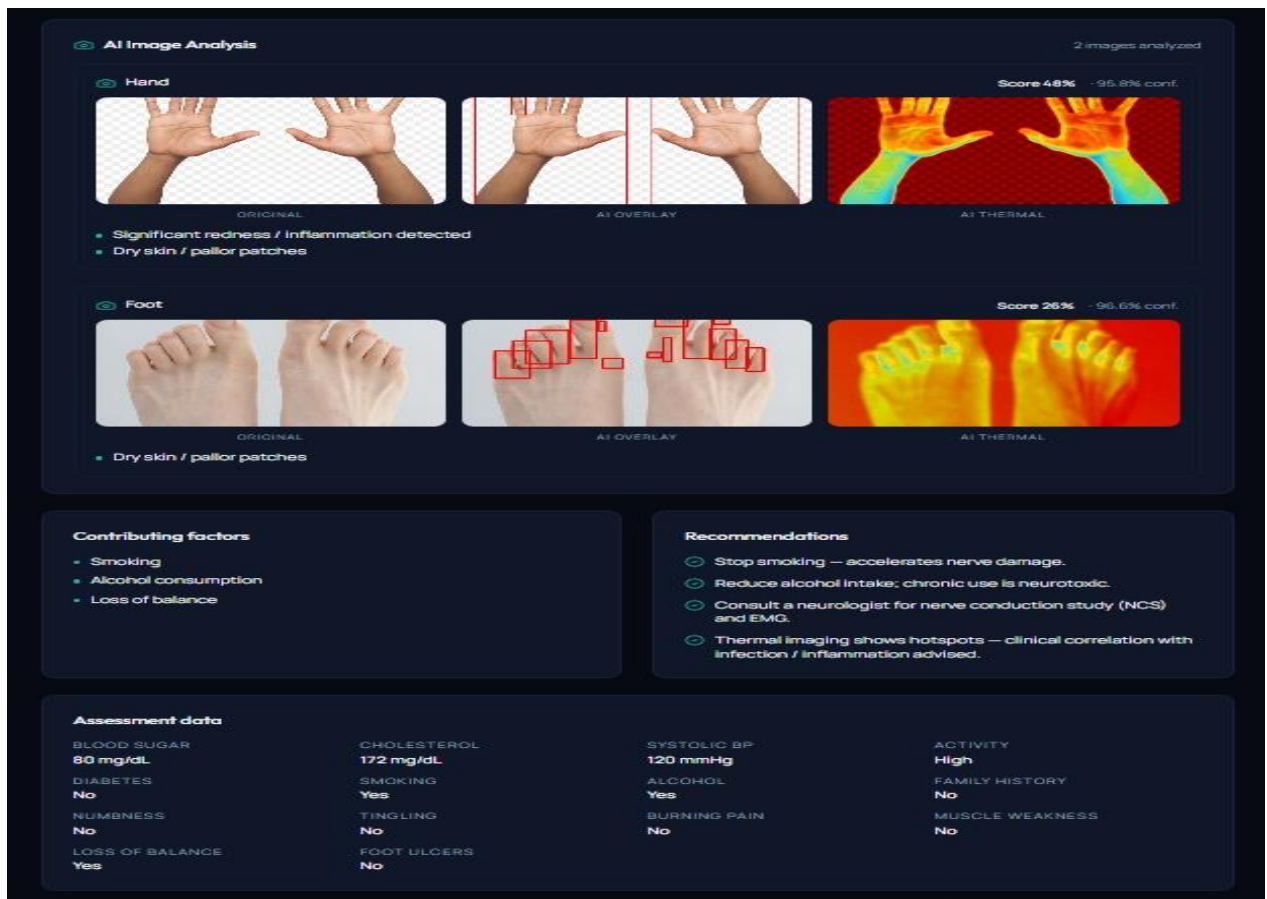
4. Prediction Page

The screenshot shows the 'New Neuropathy Prediction' form in the NeuroDetect application. The interface is dark-themed and includes a sidebar with navigation options: Dashboard, New Prediction (active), and Patients. The main form is divided into several sections:

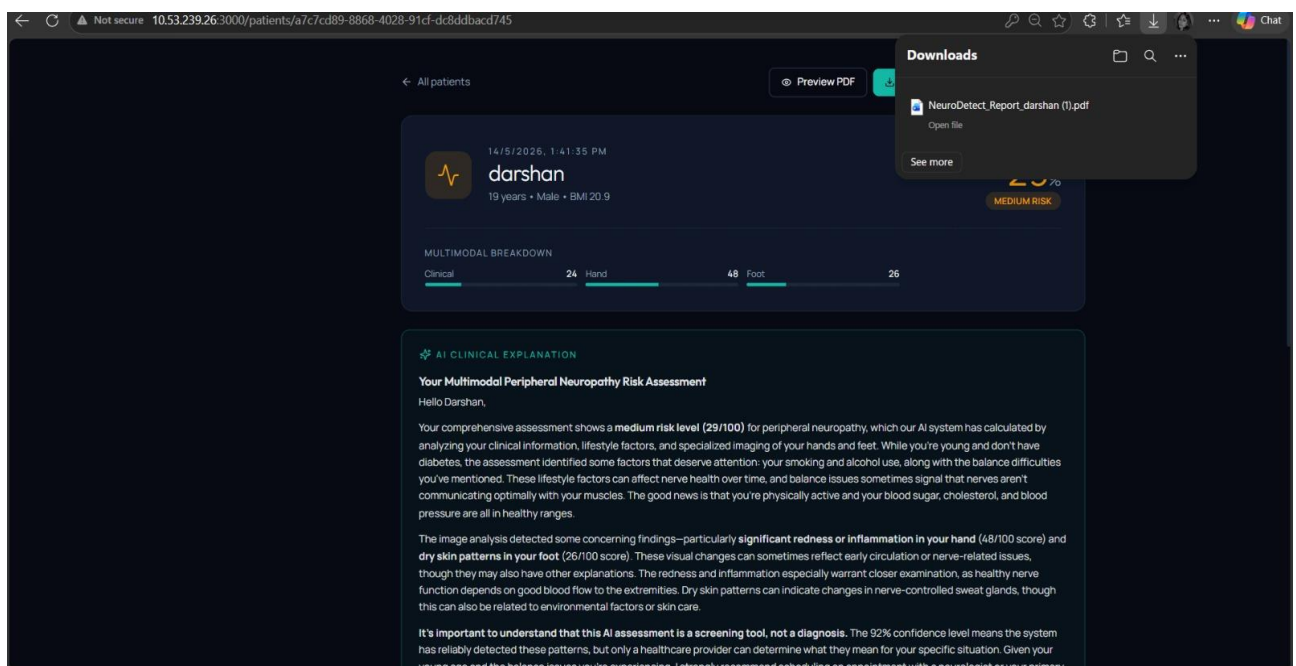
- Demographics:** Fields for Patient Name (darshan), Age (19), Gender (Male), and BMI (20.9).
- Vitals & Labs:** Fields for Blood Sugar (80), Cholesterol (172), Systolic Blood Pressure (120), and Physical Activity (High).
- Medical & Lifestyle:** Toggle switches for Diabetes, Smoking, Alcohol consumption, and Family history.
- Neurological Symptoms:** Toggle switches for Numbness, Tingling, Burning pain, Muscle weakness, Loss of balance, and Foot ulcers.
- Standard Images:** Two image uploaders for 'HAND IMAGE' and 'FOOT IMAGE'. Below each is a 'Normal' image placeholder.
- Thermal Images:** Two image uploaders for 'THERMAL HAND' and 'THERMAL FOOT'.

At the bottom, there is a status bar indicating '2 images attached - multimodal analysis enabled', a 'Reset' button, and a 'Run Multimodal AI Prediction' button. The browser address bar shows '10.53.239.26:3000/new'.

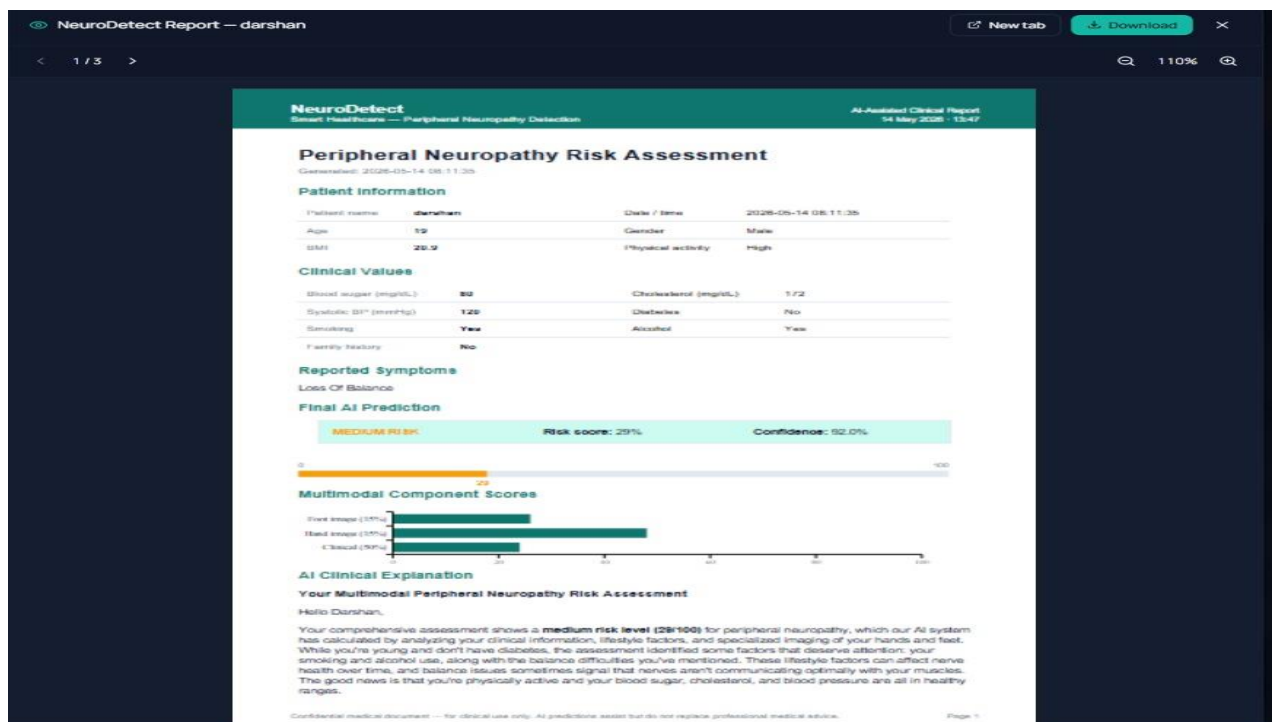
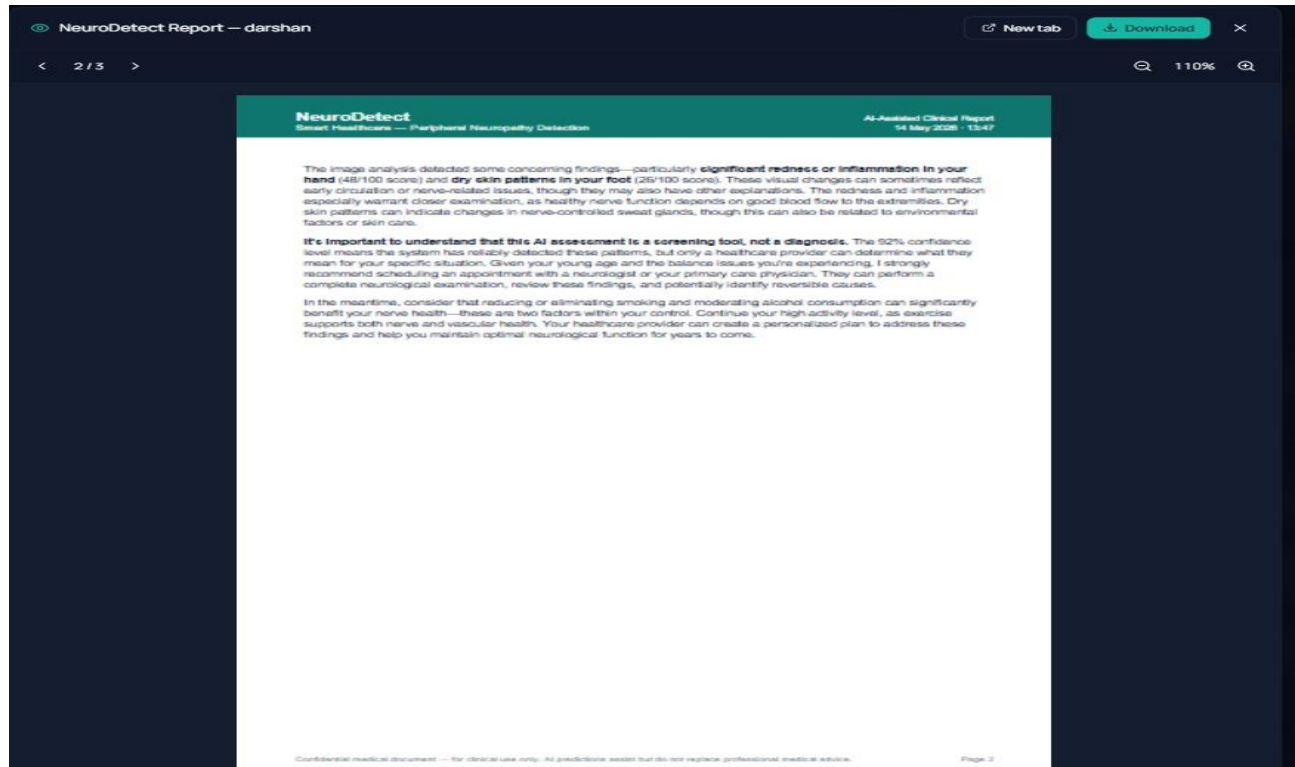
5. Thermal Image Analysis

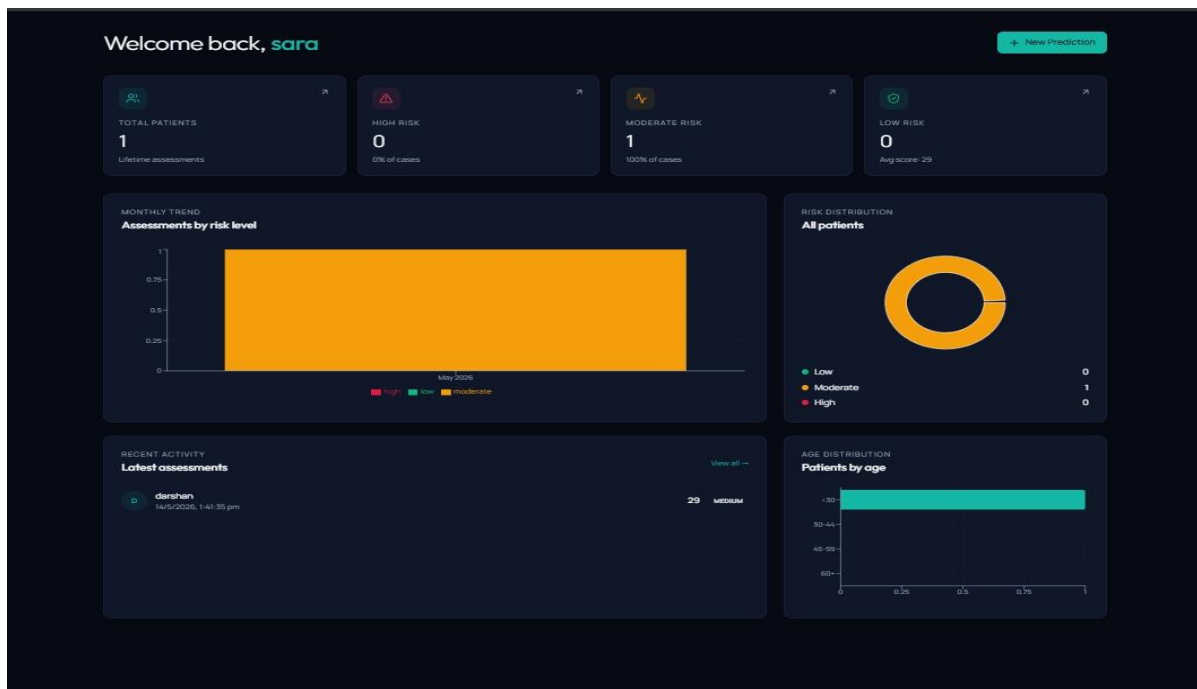
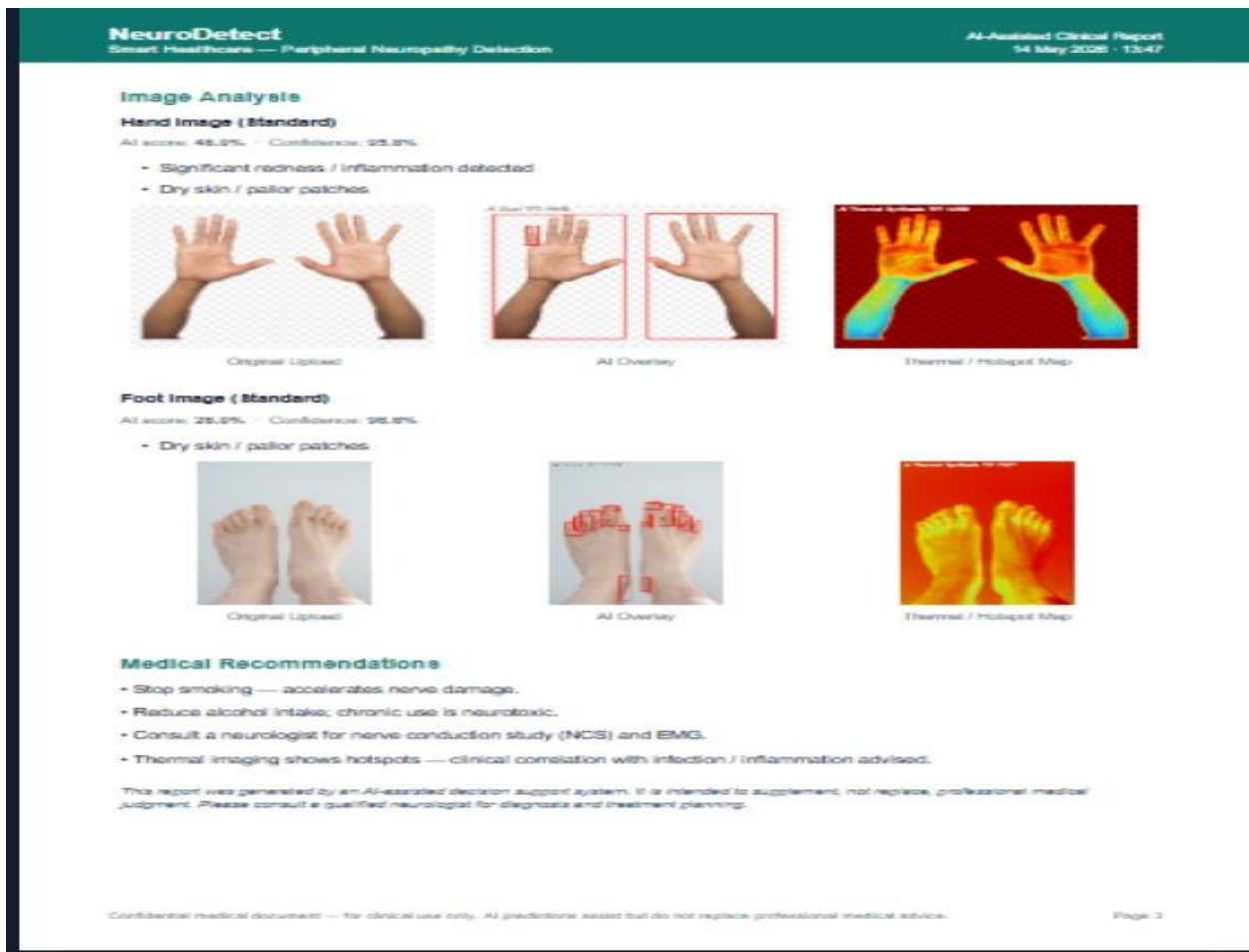


6. Medium Risk Level

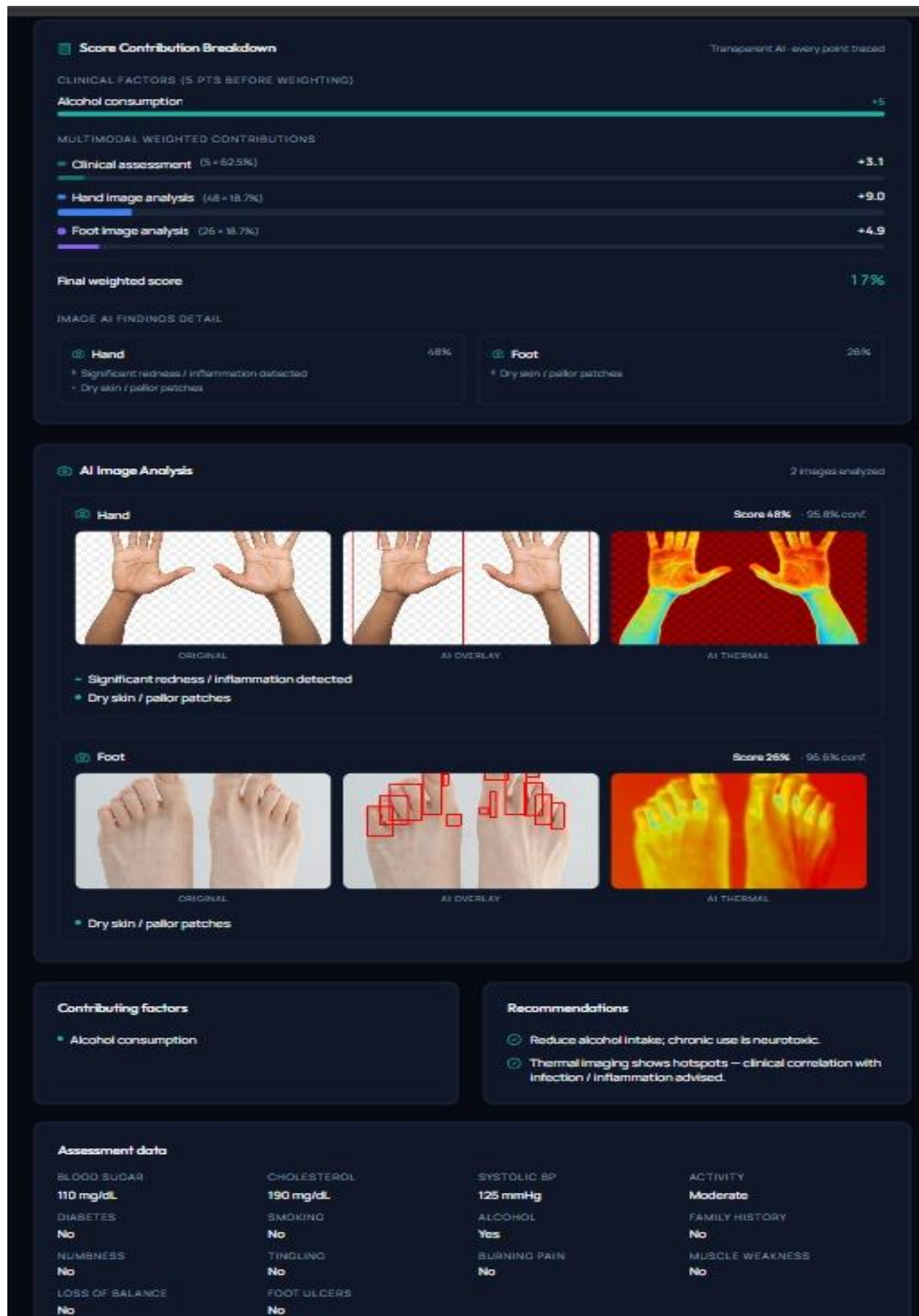


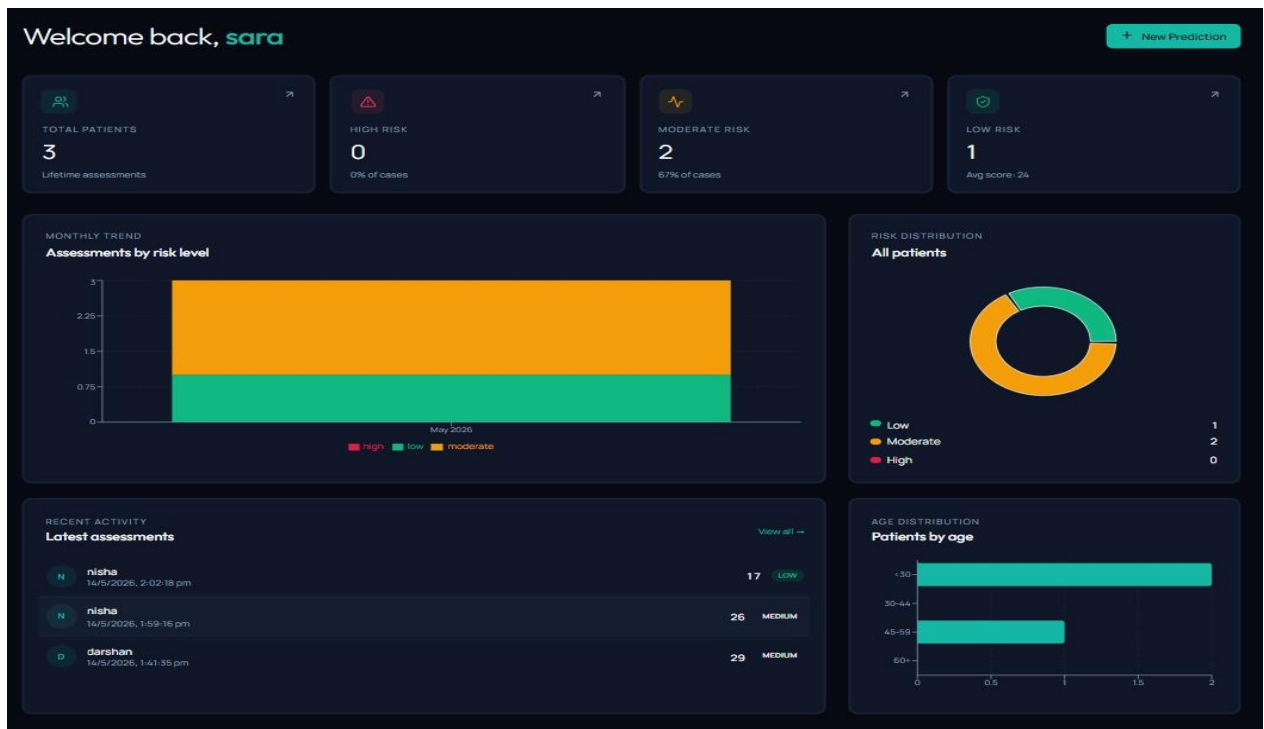
7.Pdf Report





8.Updated Dashboard





10.High Risk Level

New Neuropathy Prediction

Demographics

Basic patient information

PATIENT NAME: AGE:

GENDER: BMI (KG/M²):

Vitals & Labs

Recent measurements

BLOOD SUGAR (MG/DL): CHOLESTEROL (MG/DL):

SYSTOLIC BLOOD PRESSURE (MMHG): PHYSICAL ACTIVITY:

Medical & Lifestyle

Tap to toggle

Diabetes: ☒ Smoking: ☐

Alcohol consumption: ☒ Family history: ☐

Neurological Symptoms

Tap to toggle observed symptoms

Numbness: ☒ Tingling: ☒

Burning pain: ☒ Muscle weakness: ☐

Loss of balance: ☒ Foot ulcers: ☐

Standard Images

Phone/camera photos. AI generates pseudo-thermal visualization.

HAND IMAGE: ☒ Remove FOOT IMAGE: ☒ Remove

normal hand2.jpeg 134 KB Tap to replace

normal leg image.jpeg 24 KB Tap to replace

Thermal Images

Real FLIR / thermal camera images for hotspot & asymmetry analysis.

THERMAL HAND: Real thermal image

THERMAL FOOT: Real thermal image

2 images attached – multimodal analysis enabled

Reset Run Multimodal AI Prediction

[← All patients](#)
[Preview PDF](#)
[Download PDF Report](#)
[Delete](#)

14/5/2026, 2:09:58 PM

sumanth

51 years • Male • BMI 24.5

RISK

62%

HIGH RISK

MULTIMODAL BREAKDOWN

Clinical 82 Hand 32 Foot 26

AI CLINICAL EXPLANATION

Your Peripheral Neuropathy Risk Assessment Results

Hello Sumanth, thank you for completing this comprehensive assessment. The multimodal analysis—which combines your medical history, symptoms, and specialized imaging of your hands and feet—indicates a **high risk score of 62/100** for peripheral neuropathy. This means the assessment has identified several concerning factors that warrant prompt medical attention, though this is not a diagnosis but rather a recommendation for further evaluation by a specialist.

Why These Factors Matter

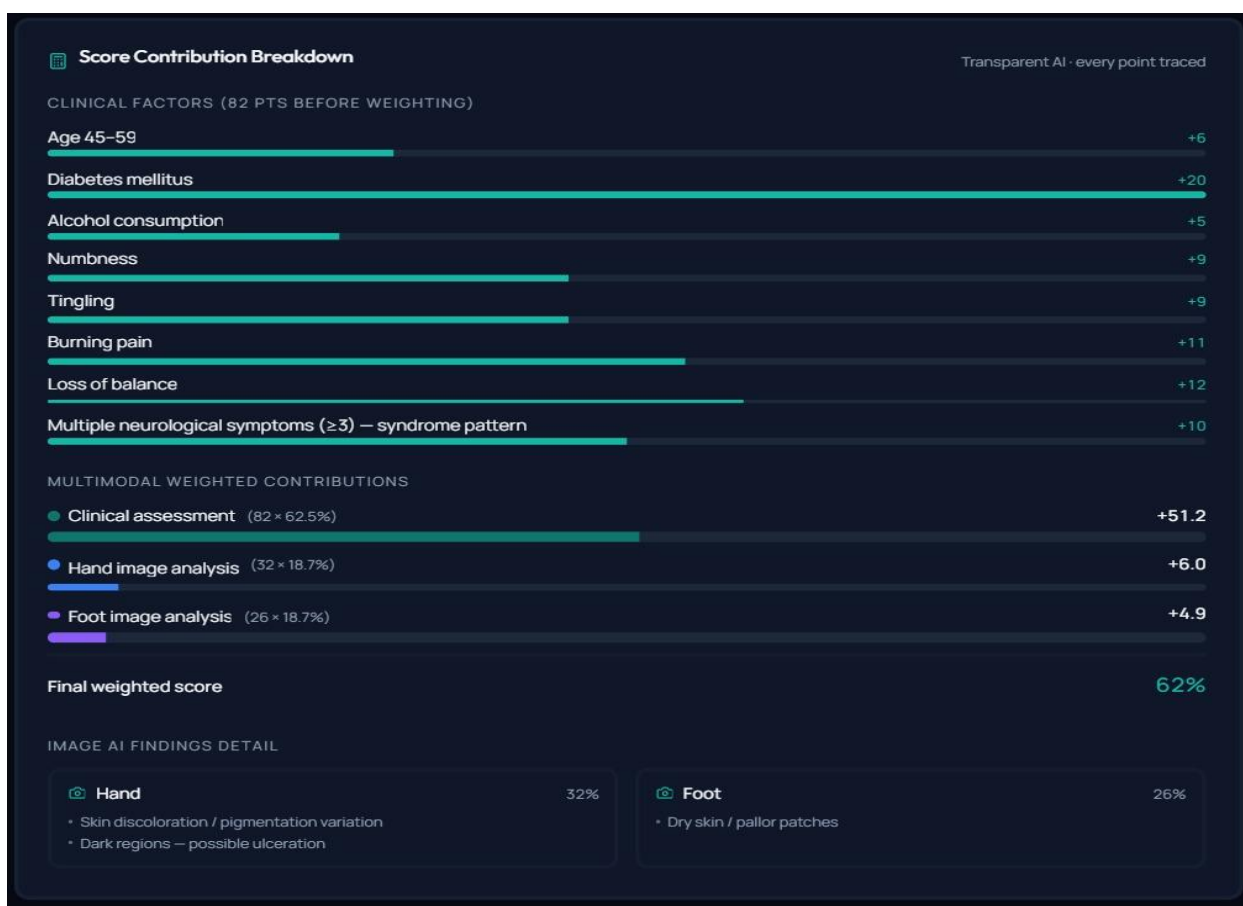
Your results show a combination of important risk factors and active symptoms. At 51 with diabetes and regular alcohol consumption, you have two significant risk factors that can affect nerve health over time. More importantly, you're experiencing a pattern of four neurological symptoms—numbness, tingling, burning pain, and balance difficulties—which often occur together in peripheral neuropathy. The image analysis detected skin changes on your hands (discoloration and darker regions) and dryness on your feet that may reflect reduced circulation or early nerve-related changes. While your blood sugar is currently well-controlled at 110 mg/dL, the presence of multiple symptoms suggests your nerves may already be sending warning signals that need professional evaluation.

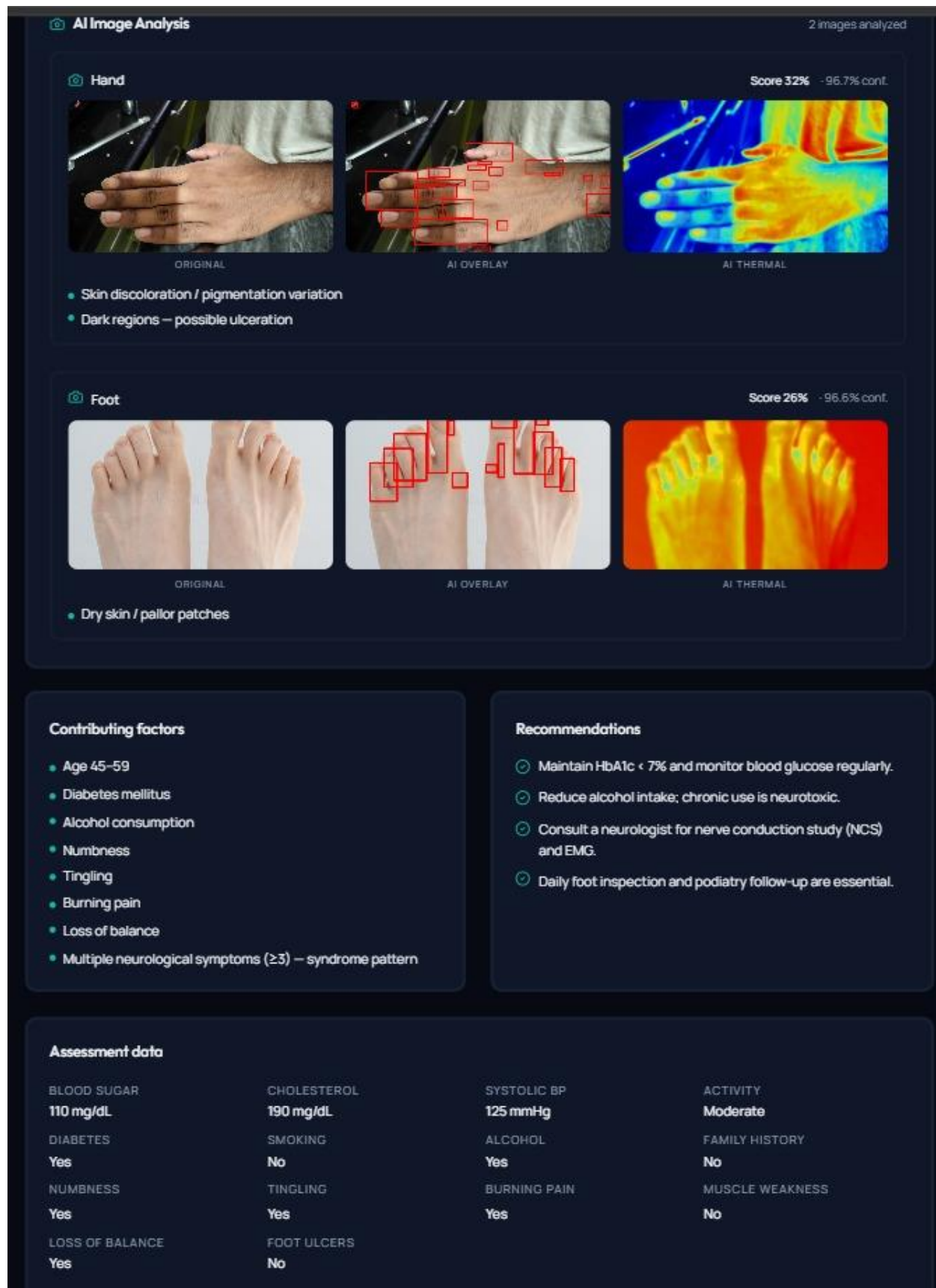
What This Assessment Means

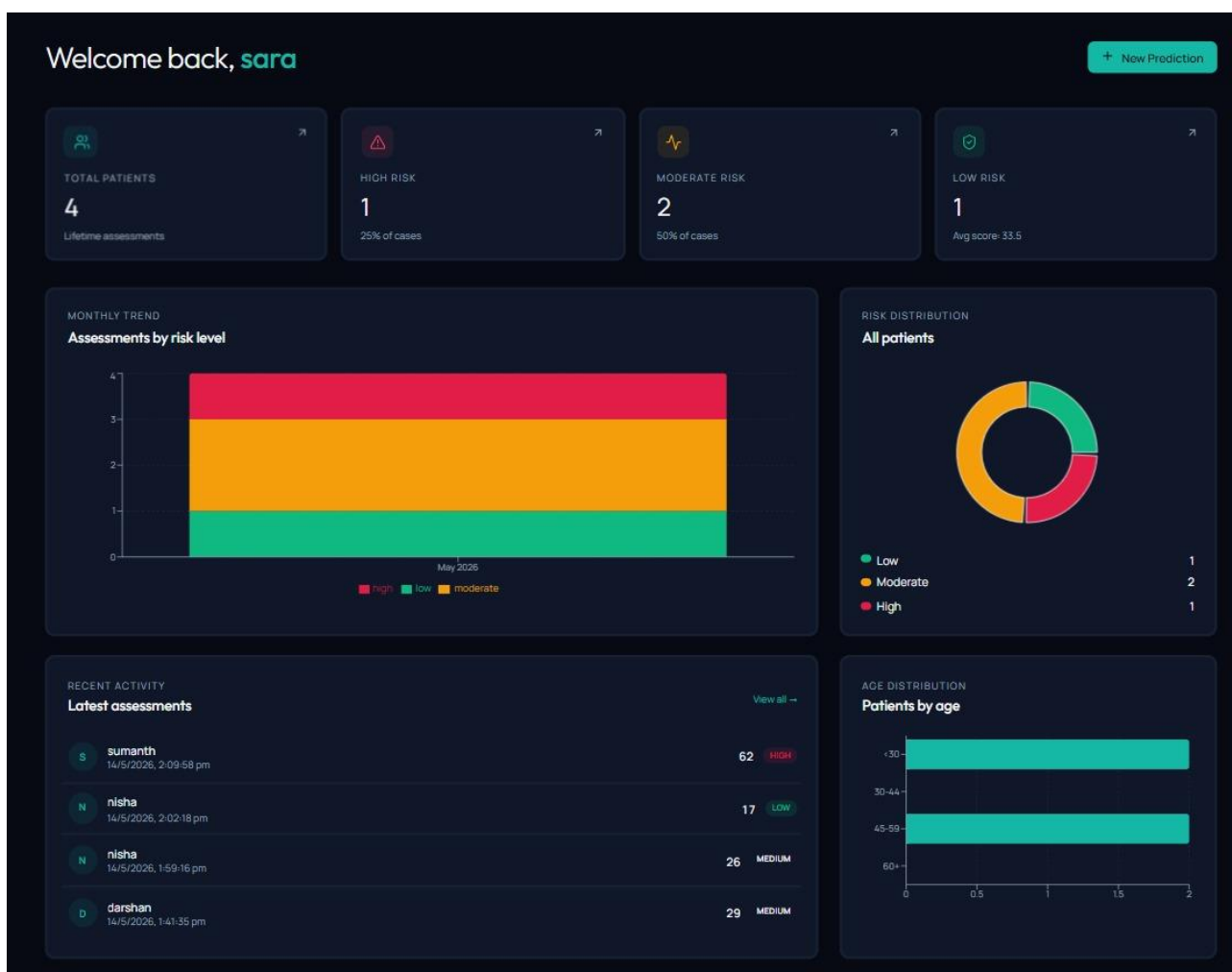
The 90.7% confidence level indicates the AI system found consistent patterns across your clinical information and imaging that align with peripheral neuropathy risk profiles. However, it's crucial to understand that only a healthcare provider can make an actual diagnosis through physical examination, nerve conduction studies, and additional testing. The skin changes detected in the images, particularly the darker regions on your hands, need in-person assessment to rule out circulation issues or other conditions requiring specific treatment.

Your Reassuring Next Steps

Please schedule an appointment with a neurologist or your primary care physician within the next 2–4 weeks to discuss these findings. Bring this assessment report with you. The good news is that early detection creates opportunities for intervention—many people successfully manage neuropathy symptoms through blood sugar optimization, lifestyle modifications, medication if needed, and monitoring. Continue your moderate physical activity, maintain careful foot care (daily inspection, moisturizing, proper footwear), and consider reducing alcohol consumption, as this can help protect nerve health. Remember, seeking specialist care now is a proactive, positive step toward maintaining your quality of life.







CHAPTER 8

CONCLUSION AND FUTURE WORK

CONCLUSION AND FUTURE WORK

Conclusion

The Smart Healthcare System for Peripheral Neuropathy Detection using Artificial Intelligence presents an efficient, non-invasive, and scalable approach to early diagnosis and monitoring of neuropathy. By leveraging machine learning algorithms and data-driven analysis, the system can identify patterns and abnormalities in patient data that may not be easily detected through traditional clinical methods. This improves the chances of early intervention, which is critical in preventing the progression of nerve damage and associated complications.

The integration of AI enhances diagnostic accuracy, reduces dependency on manual examination, and supports healthcare professionals in making faster and more informed decisions. Additionally, the system promotes accessibility by enabling remote screening and continuous monitoring, which is particularly beneficial in areas with limited medical resources.

Overall, this project demonstrates the potential of combining healthcare and artificial intelligence to build intelligent diagnostic tools that are efficient, reliable, and patient-centric. Future enhancements may include expanding the dataset, integrating real-time sensor data, and improving model accuracy to further strengthen the system's clinical applicability and real-world deployment.

The proposed Smart Healthcare System for Peripheral Neuropathy Detection using Artificial Intelligence provides an effective solution for early identification of neuropathy symptoms. By analyzing patient data using AI techniques, the system helps in detecting abnormalities at an early stage, which can significantly improve treatment outcomes. This approach reduces the need for frequent hospital visits and supports faster decision-making in healthcare. Overall, the system enhances the efficiency, accuracy, and accessibility of neuropathy detection.

In conclusion, the Smart Healthcare System for Peripheral Neuropathy Detection using AI successfully demonstrates how modern technology can improve medical diagnosis. The system uses intelligent algorithms to analyze symptoms and data patterns, enabling early detection of neuropathy. This not only helps doctors in diagnosis but also reduces the risk of complications through timely intervention. The project highlights the importance of AI in healthcare and its potential to transform traditional diagnostic methods into more efficient and automated systems.

Future Work

The Smart Healthcare System for Peripheral Neuropathy Detection using Artificial Intelligence has strong potential for further improvement and expansion. In the future, the system can be enhanced by integrating real-time data from wearable sensors such as temperature sensors, pressure sensors, and nerve activity monitoring devices. This would allow continuous patient monitoring instead of relying only on static input data.

Another important improvement is the use of larger and more diverse datasets collected from different age groups, genders, and medical conditions. This will help improve the accuracy and generalization of the AI model, making it more reliable in real-world clinical environments. Advanced deep learning techniques such as Convolutional Neural Networks (CNNs) or hybrid models can also be implemented to improve detection performance.

The system can further be developed into a mobile or cloud-based application, enabling remote diagnosis and telemedicine support. This would be especially useful for patients in rural or underserved areas with limited access to specialists. Additionally, explainable AI (XAI) techniques can be incorporated to make the model's predictions more transparent and understandable for healthcare professionals.

In the future, integration with hospital management systems and electronic health records (EHR) can create a more complete digital healthcare ecosystem. This will allow doctors to track patient history and monitor disease progression more effectively. Overall, future enhancements aim to make the system more accurate, scalable, user-friendly, and clinically deployable.

CHAPTER 9

REFERENCES

REFERENCES

1.Preston,F.G.,et al.(2022).

*Artificial intelligence utilising corneal confocal microscopy for the diagnosis of peripheral neuropathy in diabetes mellitus and prediabetes.***Diabetologia.**

<https://doi.org/10.1007/s00125-021-05617-x>

2.Meng,Y.,et al.(2023).

*AI-based analysis of corneal confocal microscopy images for diagnosing peripheral neuropathy.***Journal of Clinical Medicine.**

<https://doi.org/10.3390/jcm12041284>

3.Qian,Y.,et al.(2024).

*Transforming medicine:AI integration in the peripheral nervous system.***Frontiers in Neurology.**

<https://doi.org/10.3389/fneur.2024.1332048>

4.Sharma,S.K.,&Kumar,A.(2023).

*Assessment of diagnostic accuracy of wearable systems for diabetic peripheral neuropathy.***Journal of Foot and Ankle Research.**

<https://doi.org/10.1186/s13047-023-00667-3>

5.Wu,Y.,et al.(2024).

*Interpretable machine learning models for detecting diabetic peripheral neuropathy.***BMC Medical Informatics and Decision Making.**

<https://doi.org/10.1186/s12911-024-02595-z>

6.Wang,L.,et al.(2024).

*Early warning model of diabetic peripheral neuropathy using machine learning.***BMC Endocrine Disorders.**

<https://doi.org/10.1186/s12902-024-01728-9>

7.Talha,M.,et al.(2023).

*A survey of intelligent wearable systems for peripheral neuropathy diagnosis.***Technologies.**

<https://doi.org/10.3390/technologies11060163>

8.Chen,C.S.,et al.(2021).

*Chemotherapy-induced peripheral neuropathy detection via smartphone app.***JMIR mHealth and uHealth.**

<https://mhealth.jmir.org/2021/7/e27502>

9.Chitneni,A.,et al.(2022).

*Early detection of diabetic peripheral neuropathy by fMRI:Evidence-based review.***Brain Sciences.**

<https://doi.org/10.3390/brainsci12050557>

10.Haque,F.,et al.(2021).

*Machine learning approaches for diabetic neuropathy prediction.***Scientific Reports.**

<https://doi.org/10.1038/s41598-020-78787-0>

11.Lian,X.,et al.(2023).

*Risk factor analysis of diabetic peripheral neuropathy using machine learning.***BMC Medical Informatics and Decision Making.**

<https://doi.org/10.1186/s12911-023-02232-1>

12.Huang,H.W.,et al.(2020).

*Classification of chemotherapy-induced peripheral neuropathy.***Journal of Pain and Symptom Management.**

<https://doi.org/10.1155/2020/3402108>

13. Yang, H., et al. (2020).

*Early detection of diabetic neuropathy and AI-based approaches.***Frontiers in Endocrinology.**

<https://doi.org/10.3389/fendo.2019.00929>

14. Ming, A., et al. (2024).

*Game-based AI system for peripheral neuropathy assessment.***JMIR 2024.**

<https://www.jmir.org/2024/1/e52323>

15. Meng, Y., et al. (2023).

*Deep learning for peripheral neuropathy classification using corneal images.***MDPI Journal of Clinical Medicine.**

<https://doi.org/10.3390/jcm12041284>